07 搜索与索引

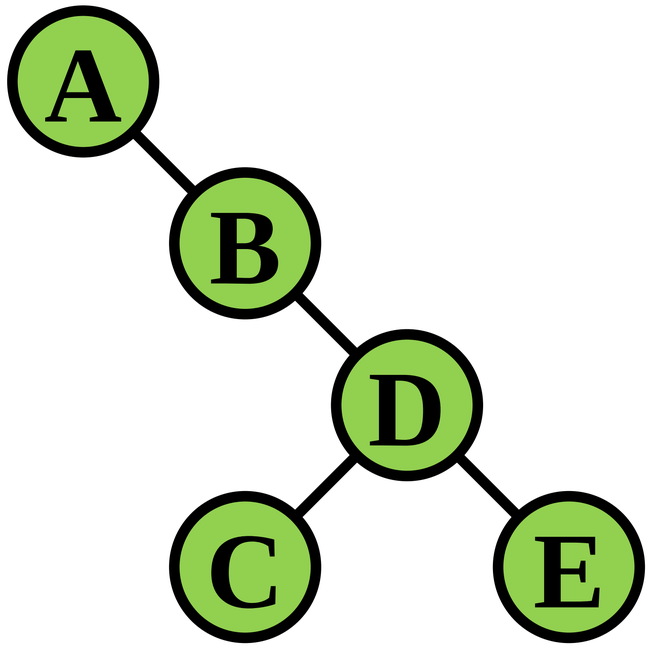
- AVL树
- Hash搜索

07 搜索与索引

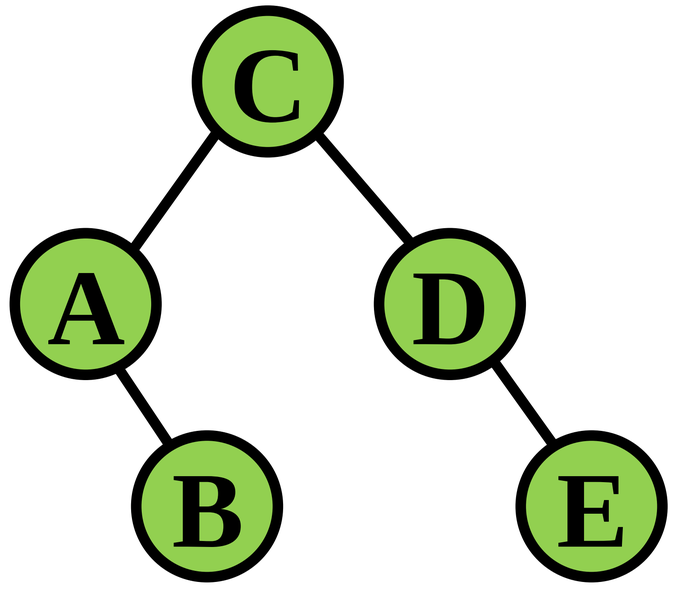
- AVL树
- Hash搜索

AVL树：高度平衡的二叉搜索树

- **一棵AVL树或者是空树，或者是具有下列性质的二叉搜索树：它的左子树和右子树都是AVL树，且左子树和右子树的高度之差的绝对值不超过1**



高度不平衡

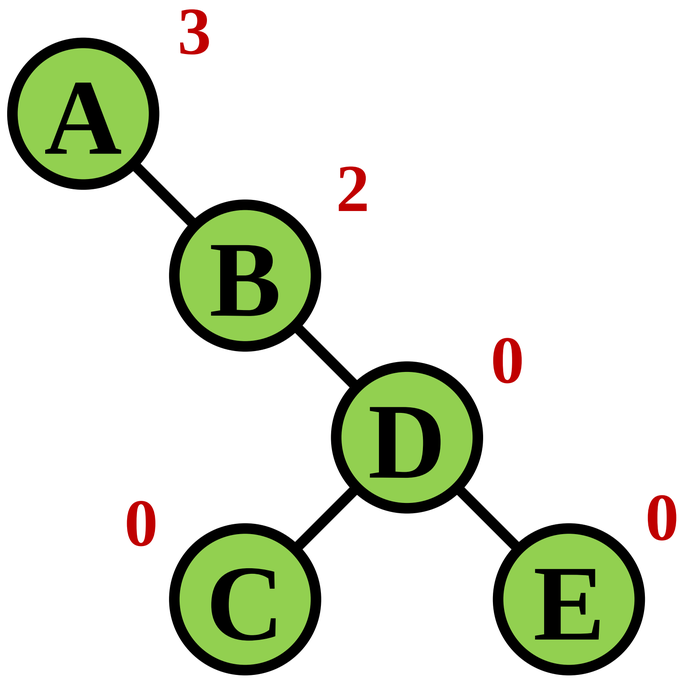


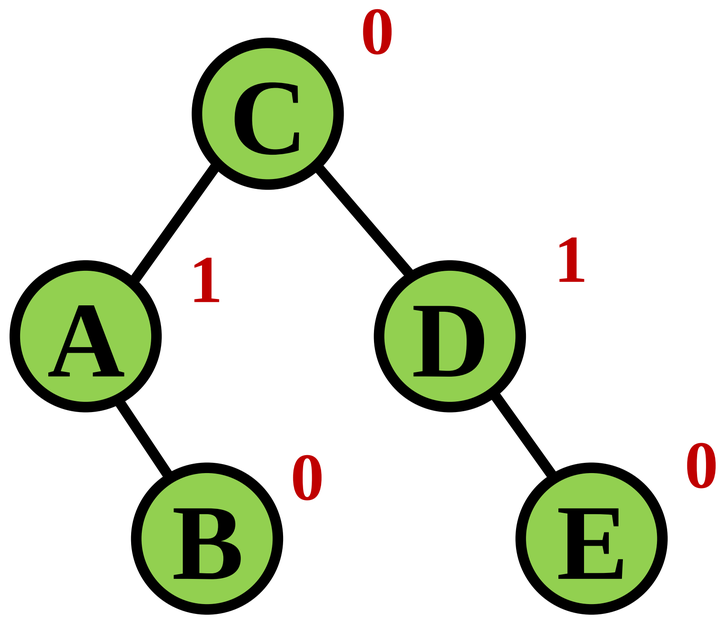
高度平衡

结点的平衡因子

右－左

- 结点的**平衡因子**(Balance Factor)：该结点右子树的高度减去左子树的高度所得的高度差
  - AVL树中任一结点平衡因子只能取-1, 0, 1
  - 如果一个结点的平衡因子的绝对值大于1，则它不是AVL树
- 如果一棵二叉搜索树是高度平衡的，且有n个结点，其高度可保持在$O(\log_2 n)$，平均搜索长度也可保持在$O(\log_2 n)$





AVL树的类定义

```
template <class Type> class AVLTree; {
template <class Type> class AVLNode {
    friend class AVLTree <Type>;
    protected:
        Type data;
        int bf; //平衡因子
        AVLNode <Type> *left, *right;
    public:
        AVLNode( ) : left(NULL), right(NULL), bf(0) { }
        AVLNode(Type d, AVLNode <Type> *l=NULL,
                AVLNode <Type> *r=NULL)
                : data(d), left(l), right(r), bf(0) { }
};
```

AVL树的类定义

```
template <class Type> class AVLTree {
   protected:
      AVLNode<Type>  *root;
      Type RefValue;
      AVLNode<Type> * Search(Type x, AVLNode<Type> *ptr);
      bool Insert(Type x, AVLNode <Type> *& ptr);
      bool Remove(Type x, AVLNode <Type> *& ptr);
      void RotateL(AVLNode<Type> *& ptr);
      void RotateR(AVLNode<Type> *& ptr);
      void RotateLR(AVLNode<Type> *& ptr);
      void RotateRL(AVLNode<Type> *& ptr);
      int Height(AVLNode<Type> * ptr) const;
```

AVL树的类定义

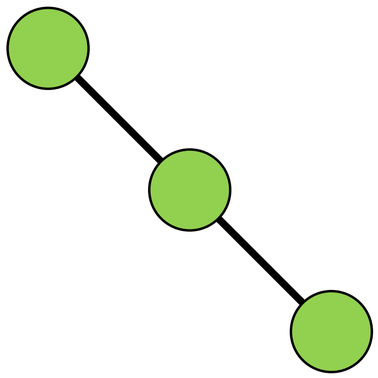
```
public:
    AVLTree( ) : root(NULL) { }
    AVLTree(Type value) : RefValue(value), root(NULL) { }
    bool Search(Type x) { return Search(x, root) != NULL; }
    bool Insert(Type x)  { return Insert(x, root); }
    bool Remove(Type x)  { return Remove(x, root); }
    friend istream& operator >> (istream& in, AVLTree<Type> & Tree);
    friend ostream& operator << (ostream& out, AVLTree<Type>& Tree);
    int Height( ) const { return Height(root); }
}
```

平衡化旋转

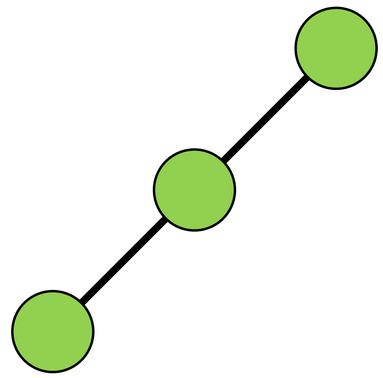
- 在一棵AVL树中插入一个新结点，会改变相关结点的平衡因子，如果造成了不平衡，就必须调整树的结构，使之平衡化，即**平衡化旋转**
  - 单旋转(包括左单旋转和右单旋转)
  - 双旋转(包括先左后右双旋转和先右后左双旋转)

- 在插入一个新结点后，需要**从插入位置沿通向根的路径回溯**，检查各结点的平衡因子，如果在某一结点n发现高度不平衡，停止回溯，从结点n起，沿刚才回溯的路径取直接下两层的结点s和t 总共3个结点
  - 如果n, s, t处于一条直线上，则采用单旋转进行平衡化
  - 如果n, s, t处于一条折线上，则采用双旋转进行平衡化

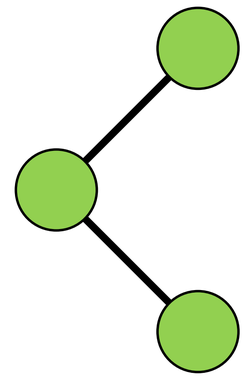
4种需要旋转的情况



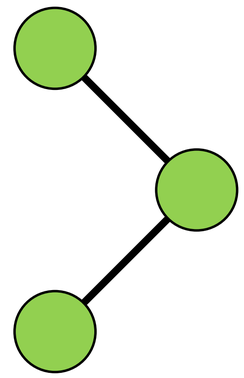
左单旋转



右单旋转



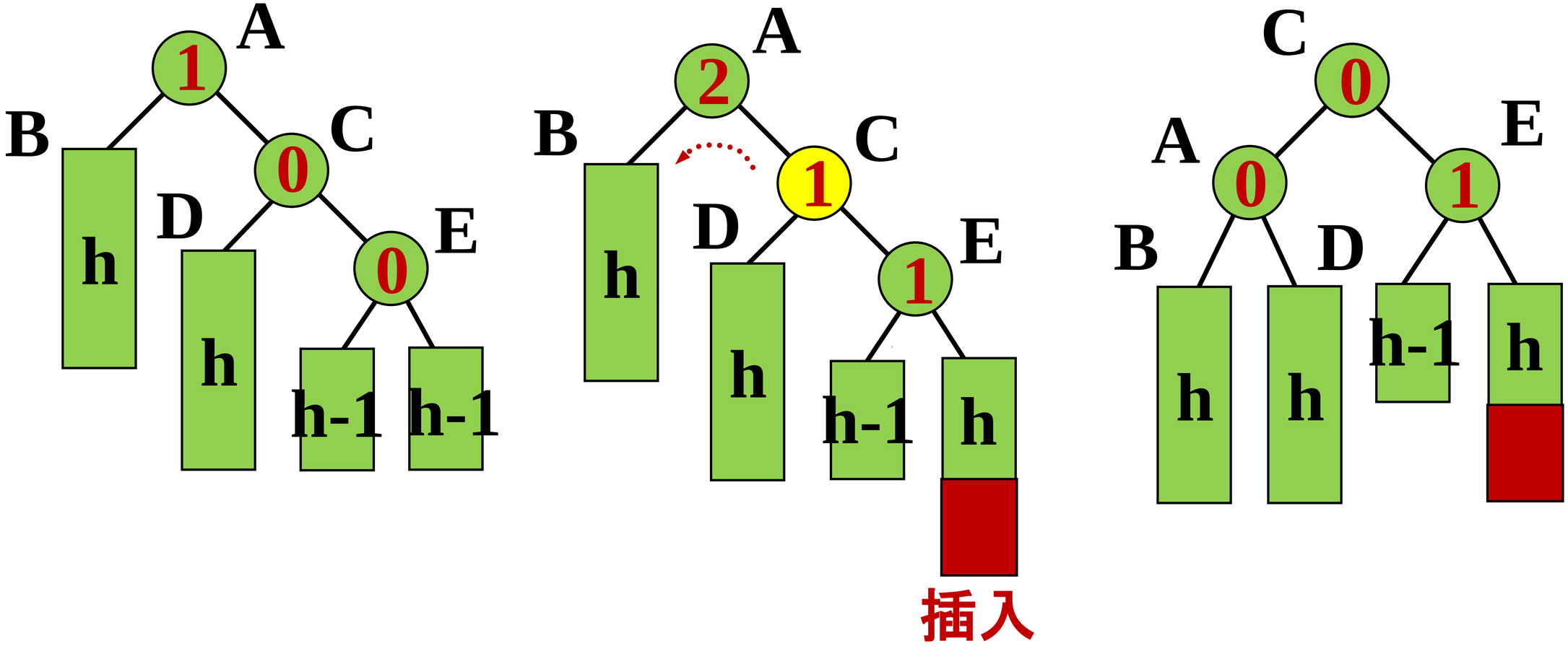
先左后右
双旋转



先右后左
双旋转

左单旋转

- 在结点A的右子女的右子树E中插入新结点，该子树高度增1导致结点A的平衡因子变成2，造成不平衡。为使树恢复平衡，从A沿插入路径连续取3个结点A、C和E，以结点C为旋转轴，让结点A反时针旋转

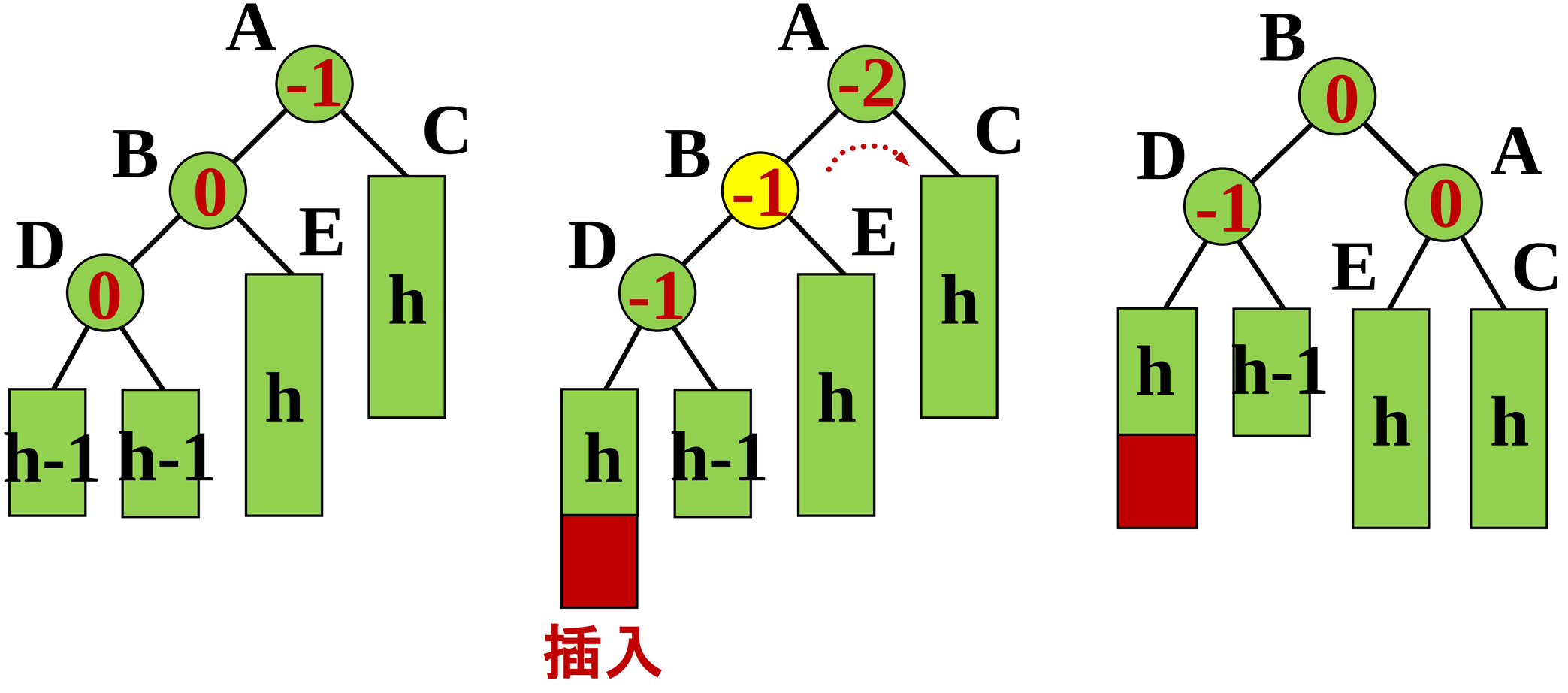


左单旋转算法

```
// 右子树比左子树高，对以ptr为根的AVL树做
// 左单旋转，旋转后新根在ptr
template <class Type> void AVLTree <Type> ::
RotateL(AVLNode <Type> *& ptr) {
    AVLNode <Type> *subL = ptr;  // 旧根，新根左子树
    ptr = subL->right;  // 新根
    subL->right = ptr->left;
    ptr->left = subL;
    ptr->bf = subL->bf = 0;
}
```

右单旋转

- 在结点A的左子女的左子树D上插入新结点，使其高度增1导致结点A的平衡因子变成-2，造成不平衡。为使树恢复平衡，从A沿插入路径连续取3 个结点A、B和D，以结点B为旋转轴，将结点A顺时针旋转

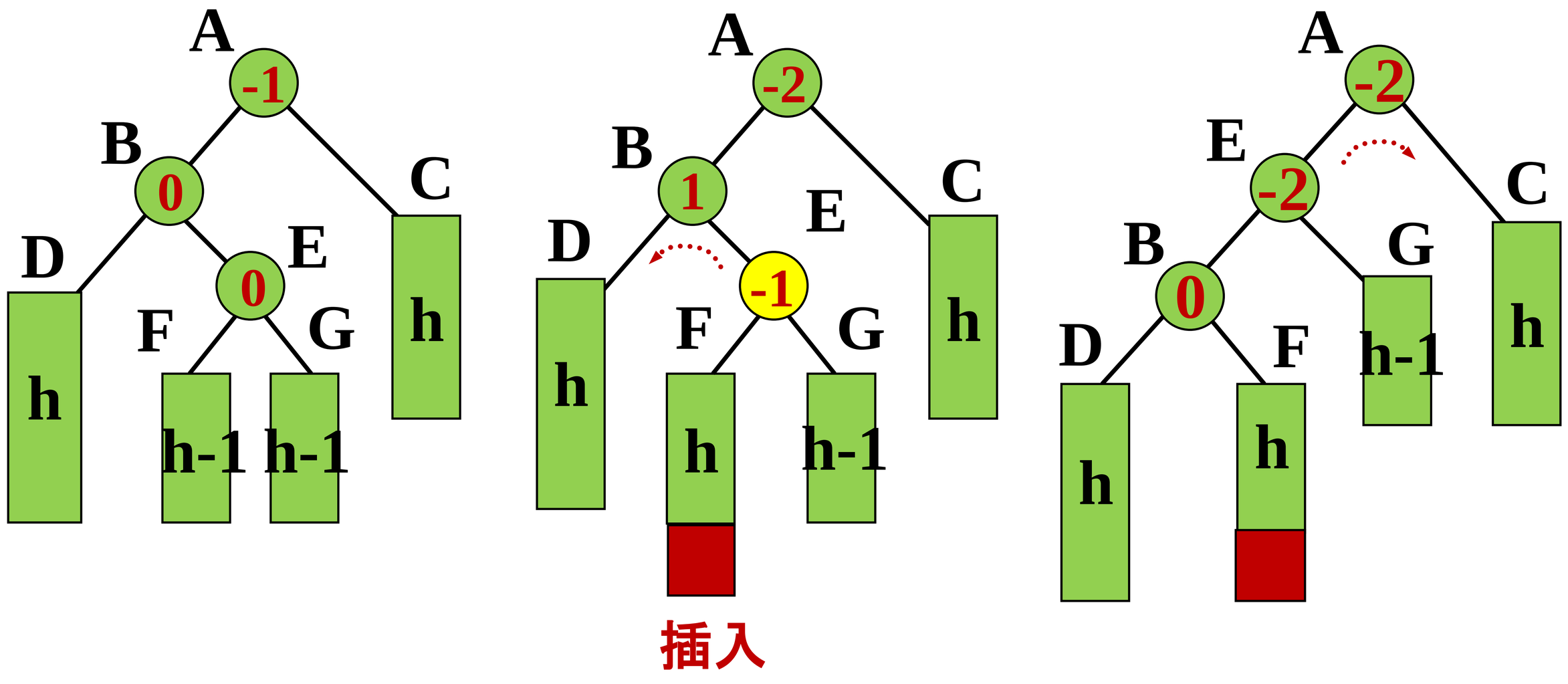


右单旋转算法

```
// 左子树比右子树高，对以ptr为根的AVL树做
// 右单旋转，旋转后新根在ptr
template <class Type> void AVLTree <Type> ::
RotateR(AVLNode <Type> * & ptr) {
    AVLNode <Type> *subR = ptr;  // 旧根，新根右子树
    ptr = subR->left;  // 新根
    subR->left = ptr->right;
    ptr->right = subR;
    ptr->bf = subR->bf = 0;
}
```

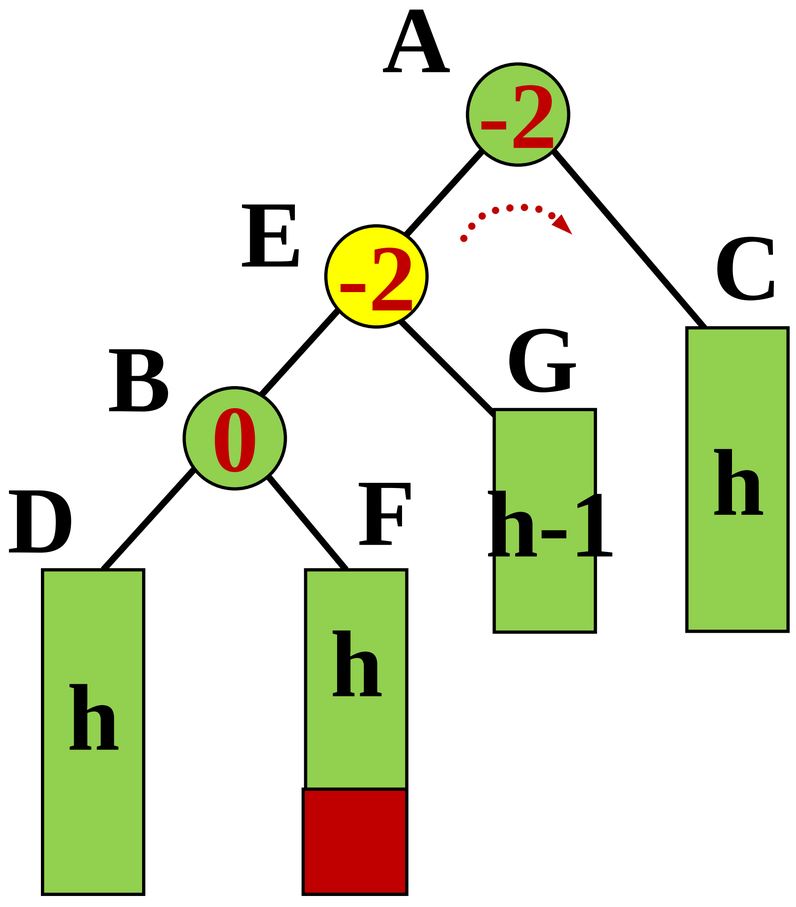
先左后右双旋转第1步

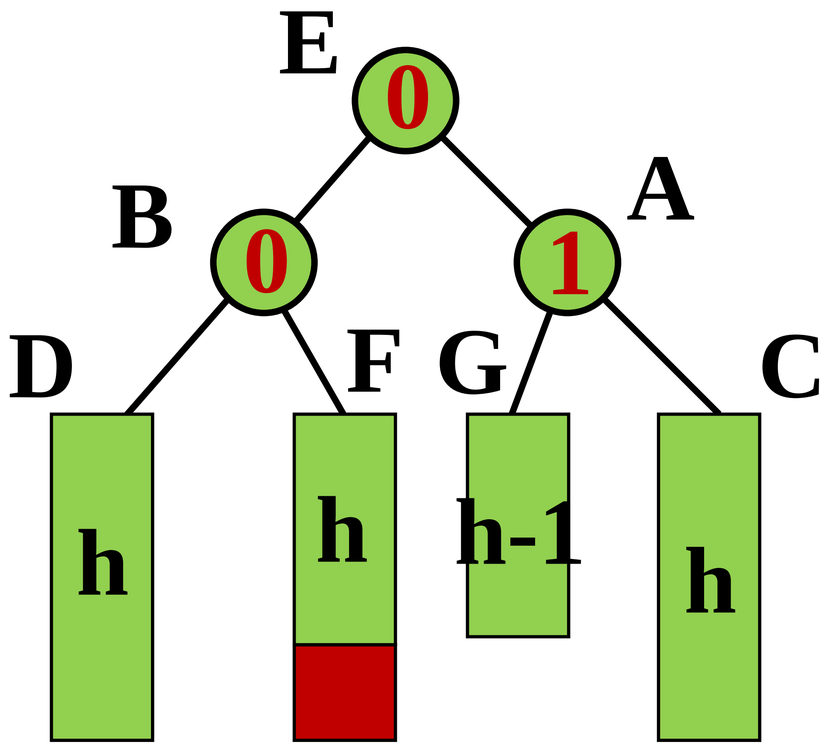
- 在结点A的左子女的右子树中插入新结点，使其高度增1导致结点A的平衡因子变成-2，造成不平衡。以结点E为旋转轴，将结点B反时针旋转，以E代替原来B的位置



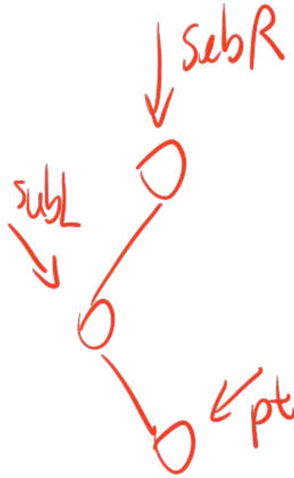
先左后右双旋转第2步

- 再以结点E为旋转轴，将结点A顺时针旋转，使之平衡化





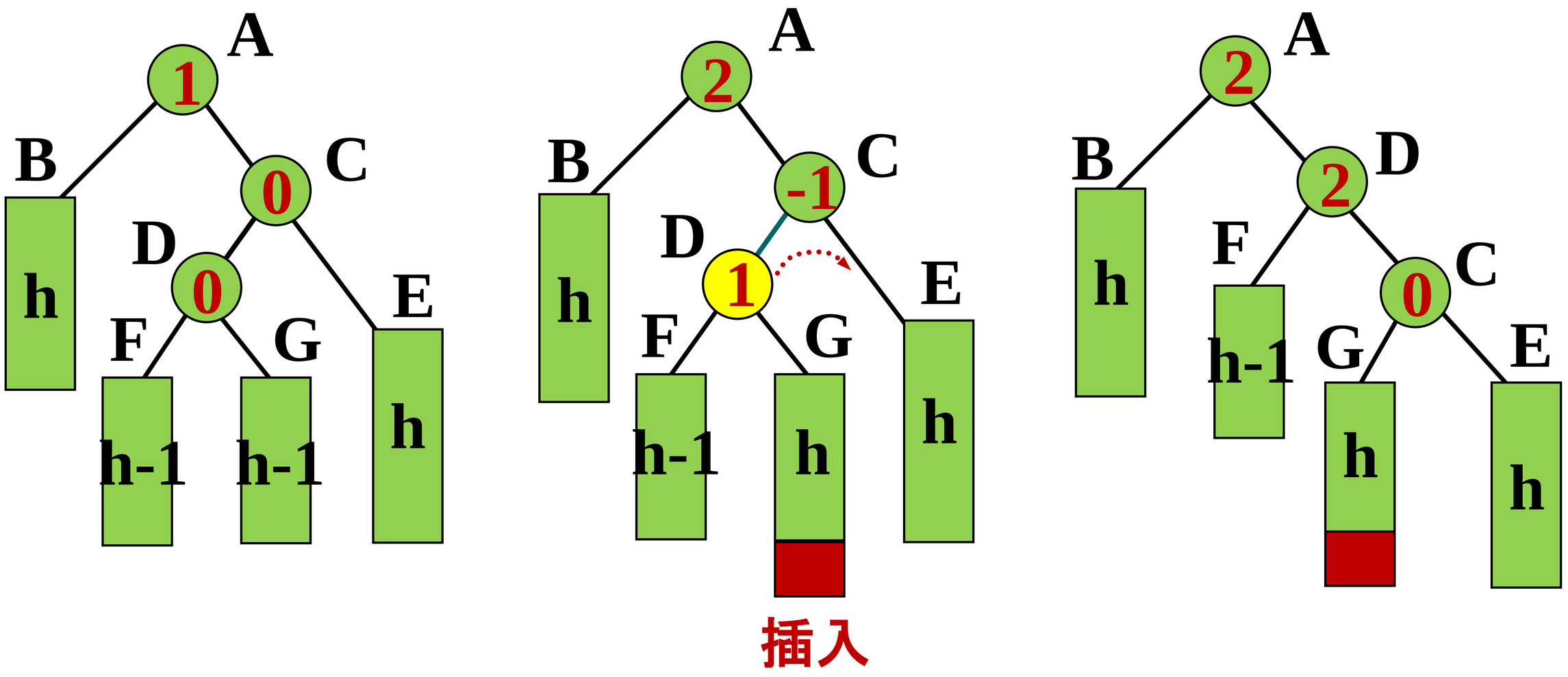
先左后右双旋转算法




```
template <class Type> void AVLTree <Type> ::
RotateLR(AVLNode <Type> * & ptr) {
    AVLNode <Type> * subR = ptr;  // 旧根，新根右子树
    AVLNode <Type> * subL = subR->left;  // 旧根左子树，新根左子树
    ptr = subL->right;  // 新根
    subL->right = ptr->left;  // 左单旋转
    ptr->left = subL;
    if (ptr->bf <= 0)  subL->bf = 0;  // 插入在左子树
    else subL->bf = -1;  // 插入在右子树
    subR->left = ptr->right;  // 右单旋转
    ptr->right = subR;
    if (ptr->bf == -1)  subR->bf = 1;  // 插入在左子树
    else subR->bf = 0;  // 插入在右子树
    ptr->bf = 0;  //新根平衡
}
```

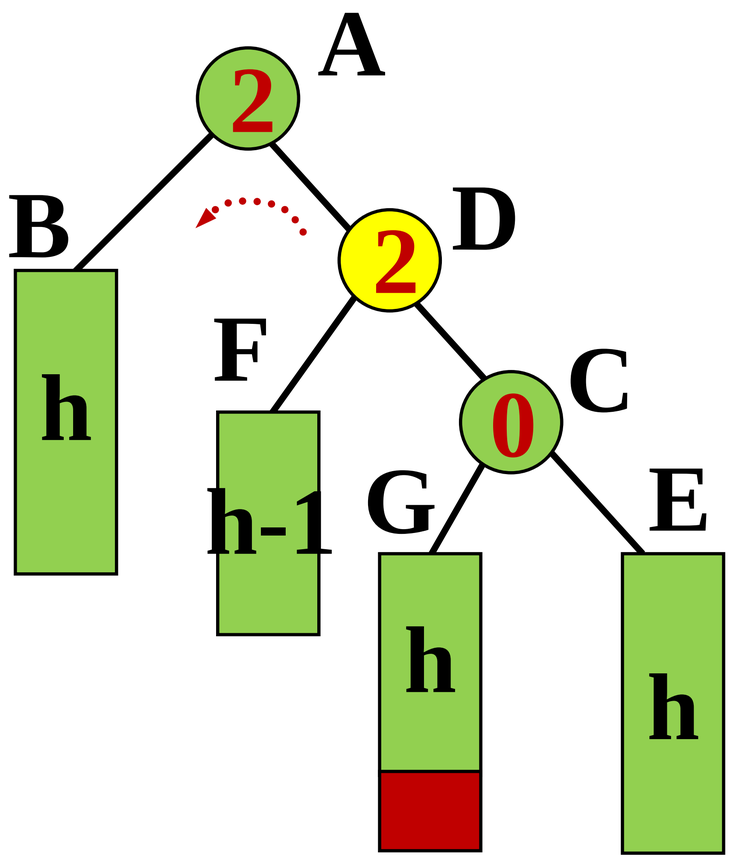
先右后左双旋转第1步

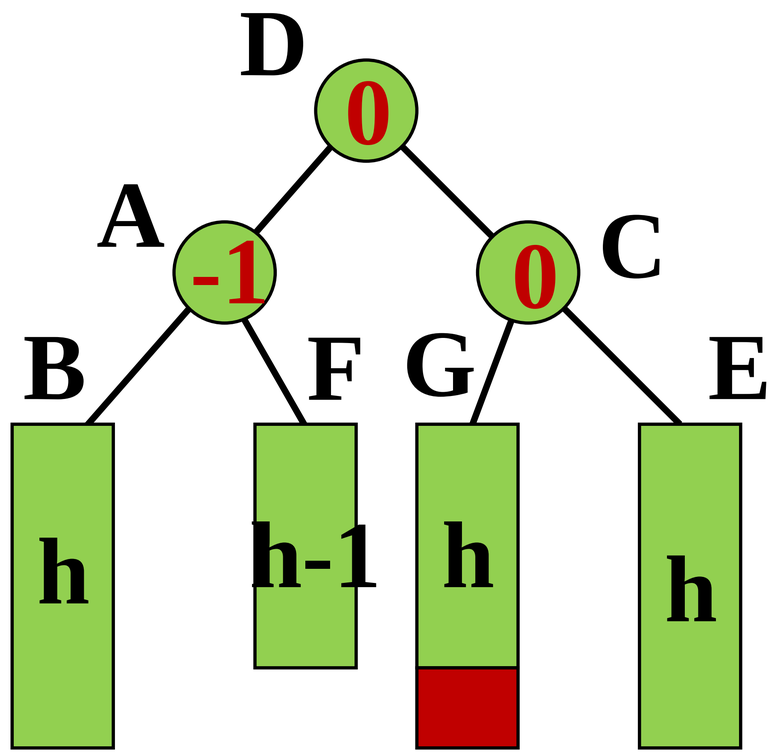
- 在结点A的右子女的左子树中插入新结点，使其高度增1导致结点A的平衡因子变成2，造成不平衡。以结点D为旋转轴，将结点C顺时针旋转，以D代替原来C的位置



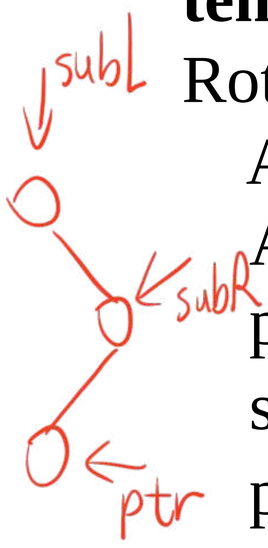
先右后左双旋转第2步

- 再以结点D为旋转轴，将结点A反时针旋转，使之平衡化





先右后左双旋转算法




```
template <class Type> void AVLTree <Type> ::
RotateRL(AVLNode <Type> * & ptr) {
    AVLNode <Type> * subL = ptr;  // 旧根，新根左子树
    AVLNode <Type> *subR = subL->right;  // 旧根右子树，新根左子树
    ptr = subR->left;  // 新根
    subR->left = ptr->right;  // 右单旋转
    ptr->right = subR;
    if (ptr->bf  >= 0)  subR->bf = 0;  // 插入在右子树
    else subR->bf = 1;  // 插入在左子树
    subL->right = ptr->left;  // 左单旋转
    ptr->left = subL;
    if (ptr->bf == 1)  subL->bf = -1;  // 插入在右子树
    else subL->bf = 0;  // 插入在左子树
    ptr->bf = 0;
}
```

AVL树的插入

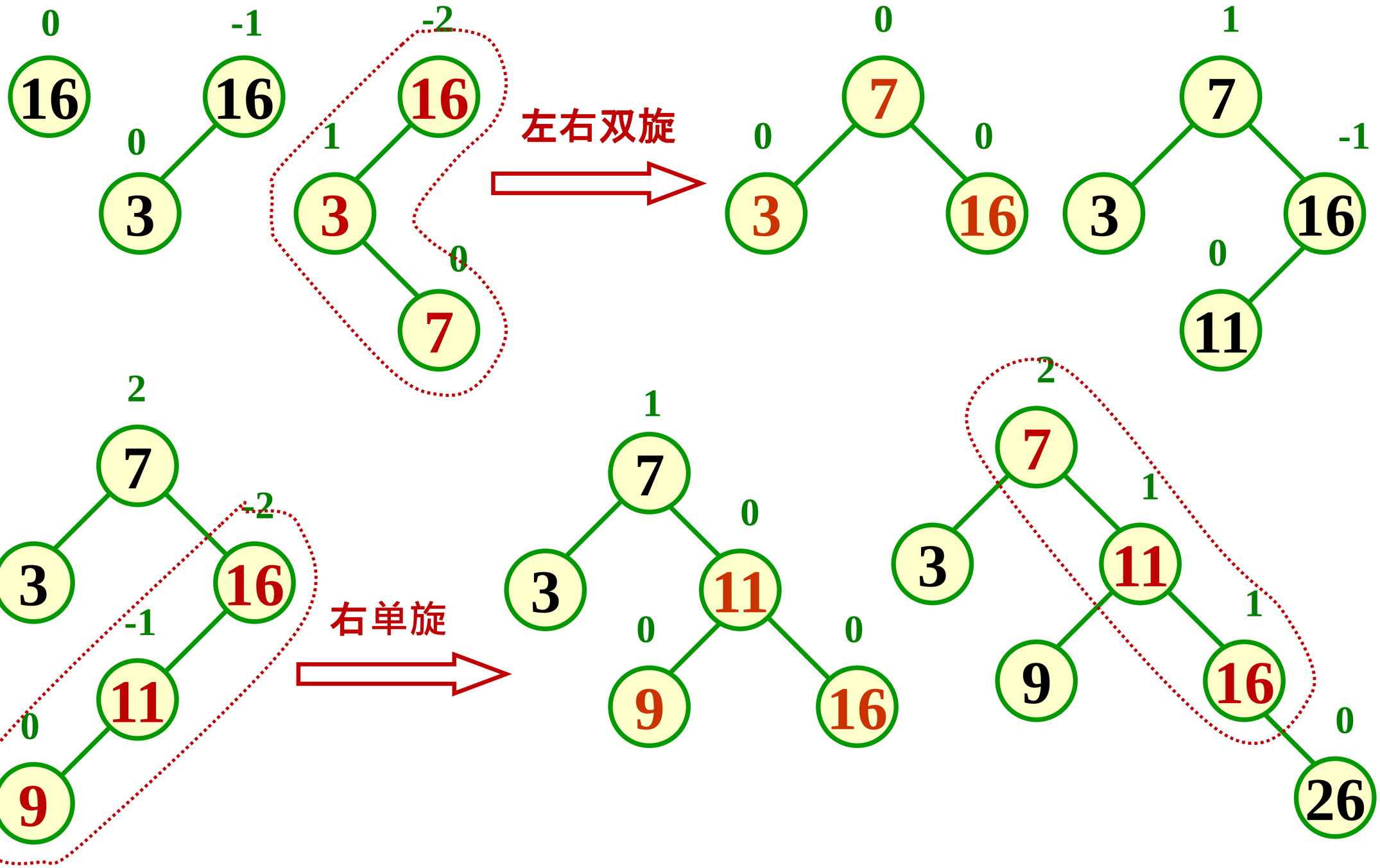
- 在向一棵本来是高度平衡的AVL树中插入一个新结点时，如果树中某个结点的平衡因子的绝对值|bf| > 1，则出现了不平衡，需要做平衡化处理
- 设新插入的结点为p，则p的平衡因子为0，从p开始向根方向回溯，调整遇到的每个结点的平衡因子
- 先考察p的父结点pr，修改后pr的平衡因子的绝对值为：
  - 为0：则p插入在 pr的矮子树上，pr高度不变，插入完成
  - 为1：则pr的高度增加但没有失去平衡，继续向根方向回溯
    - 最后到达根结点，插入完成
    - 到达某个结点s，平衡因子为0，插入完成
    - 到达某个结点s，平衡因子绝对值为2(s != pr)，则根据s的子结点的平衡因子的符号做单旋转或双旋转，旋转后，s为根的子树高度减1，不必向上回溯，插入完成

最多 调整一次即可

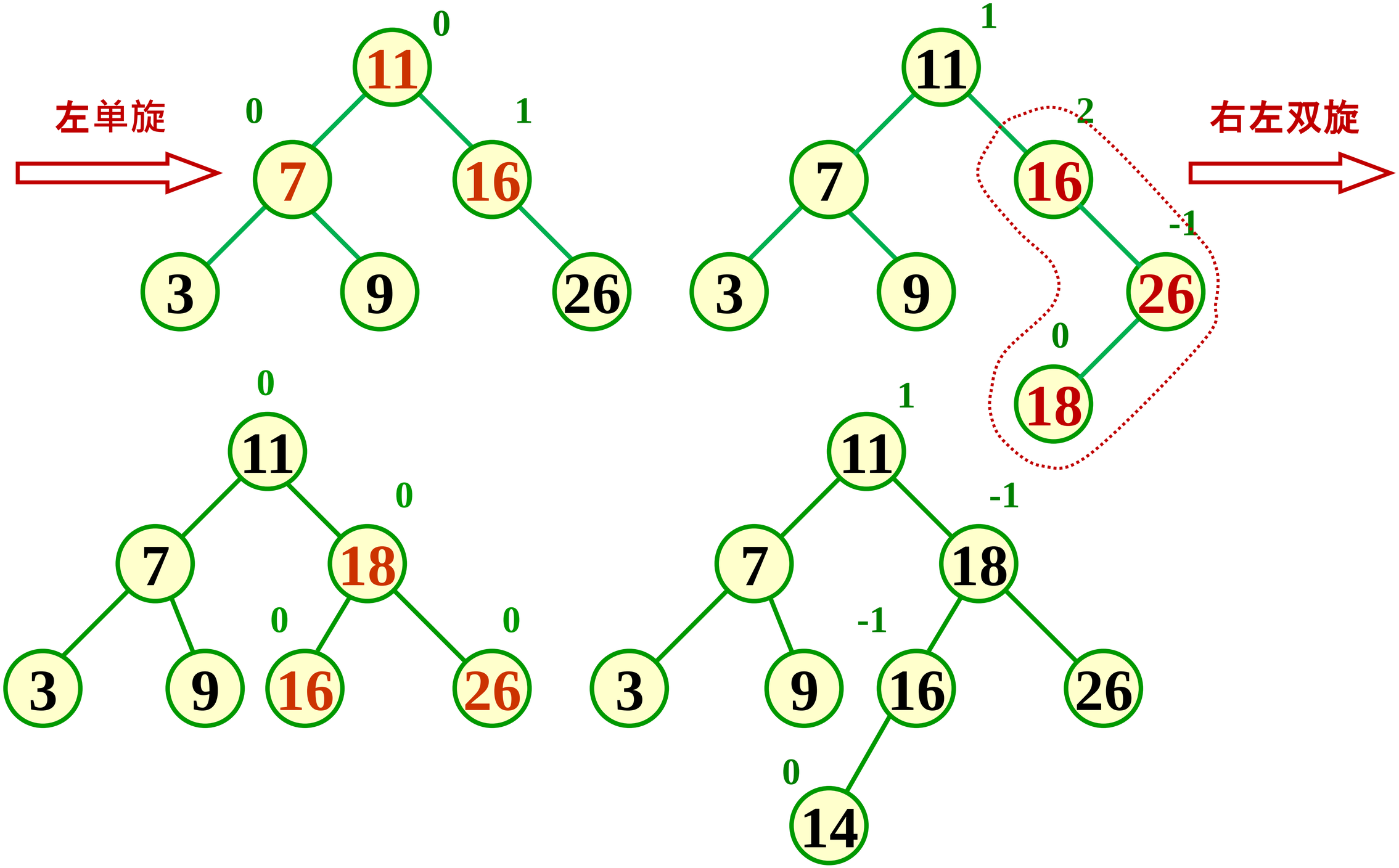
AVL树的插入算法

```
while (p != NULL) {
… // 寻找插入位置，把经过的结点压栈，循环结束时要么找到结点，返回，要么p为NULL， pr为p的父节点，pr在栈顶
}
p = new AVLNode <Type> (x);
… // 创建新结点并且连接到树上，若从空树变成一个结点的树，则返回，否则pr为父结点，p为新结点插入
while (st.IsEmpty( ) == false) {
... // 出栈，栈顶元素存在pr中， 调整pr的平衡因子， 然后判断插入结束、还是继续向根方向回溯、还是做哪一种旋转
}
if (st.IsEmpty( ) == true)  ptr = pr;  // 调整根结点
else … // 把从栈中弹出结点链接到pr
```

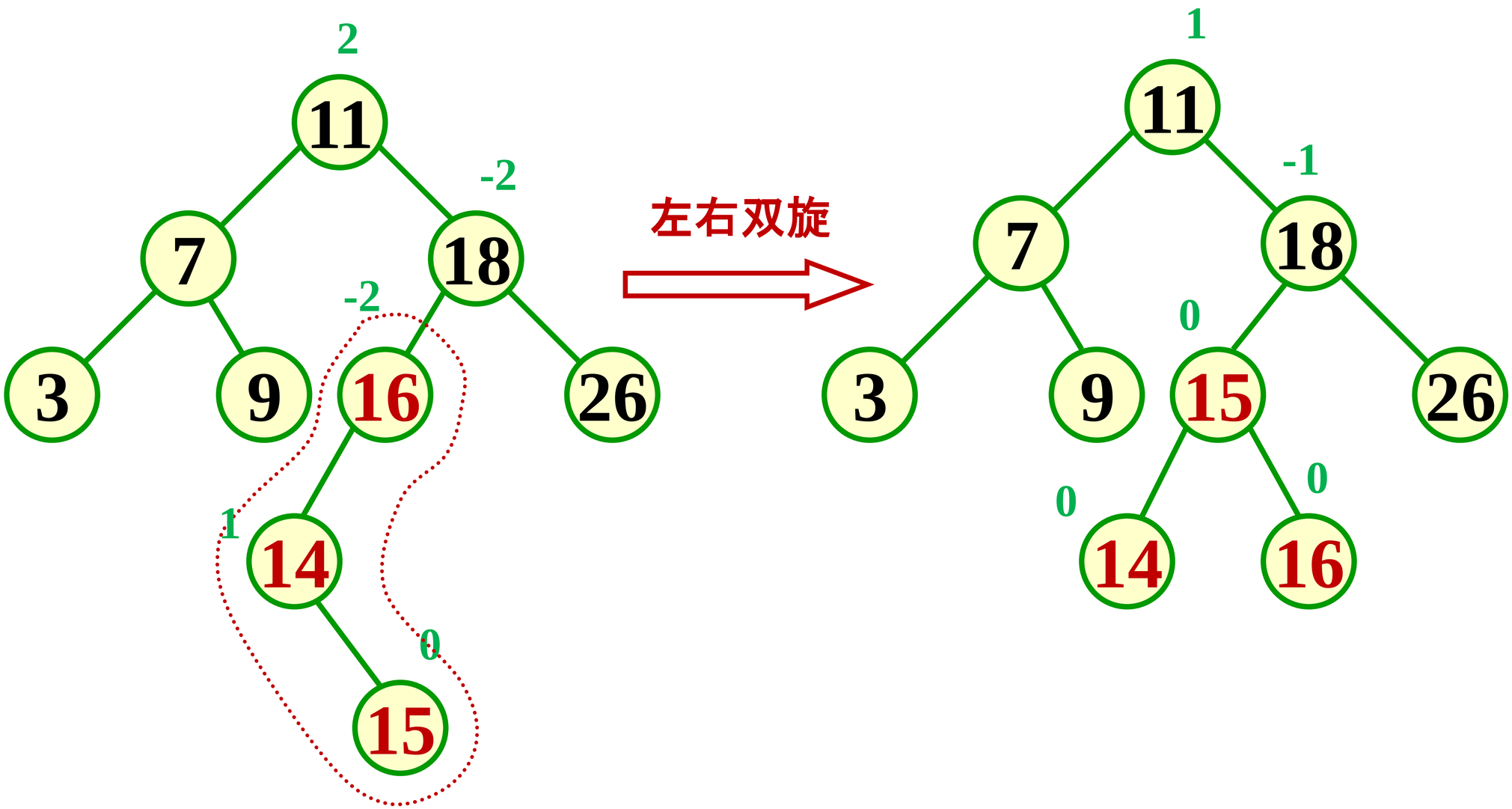
关键码序列{16, 3, 7, 11, 9, 26, 18, 14, 15}的插入和调整过程示例



关键码序列{16, 3, 7, 11, 9, 26, 18, 14, 15}的插入和调整过程示例



关键码序列{16, 3, 7, 11, 9, 26, 18, 14, 15}的插入和调整过程示例

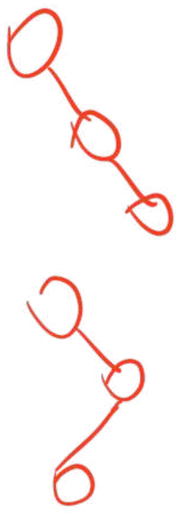


AVL树的删除

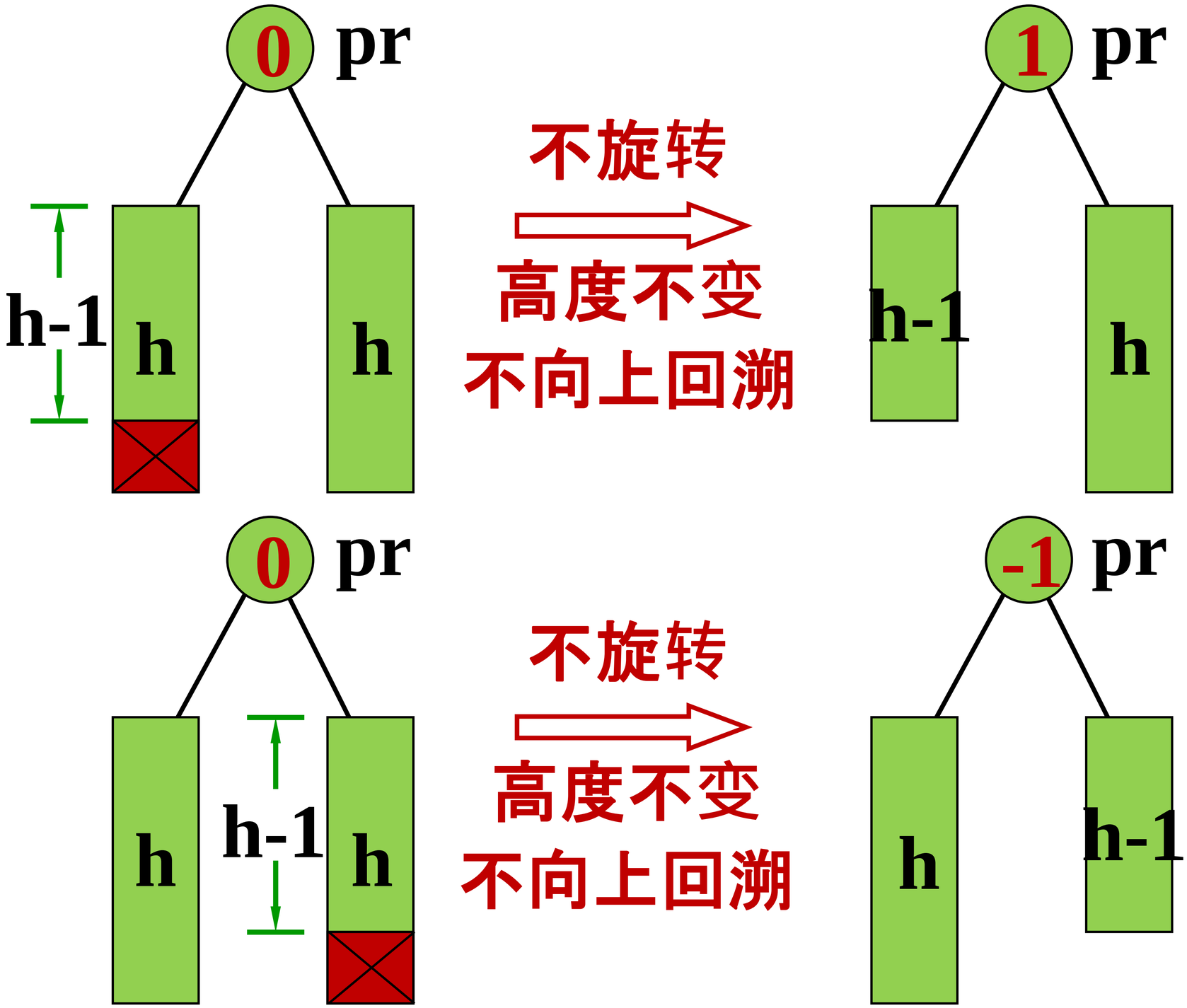
- 情况1：如果被删结点p最多只有一个子女
  - 将p从树中删除
  - 由于p最多有一个子女，把p的父结点pr中原来指向p的指针改成指向p的子女；如果结点p没有子女，则pr的相应指针置为NULL
- 情况2：如果被删结点p有两个子女
  - 搜索p在中序下的直接前驱q(也可以找直接后继)
  - 把q的内容复制到p中，再把q当作被删结点，而q最多有一个子女，转化为情况1的方法进行删除

AVL树的删除

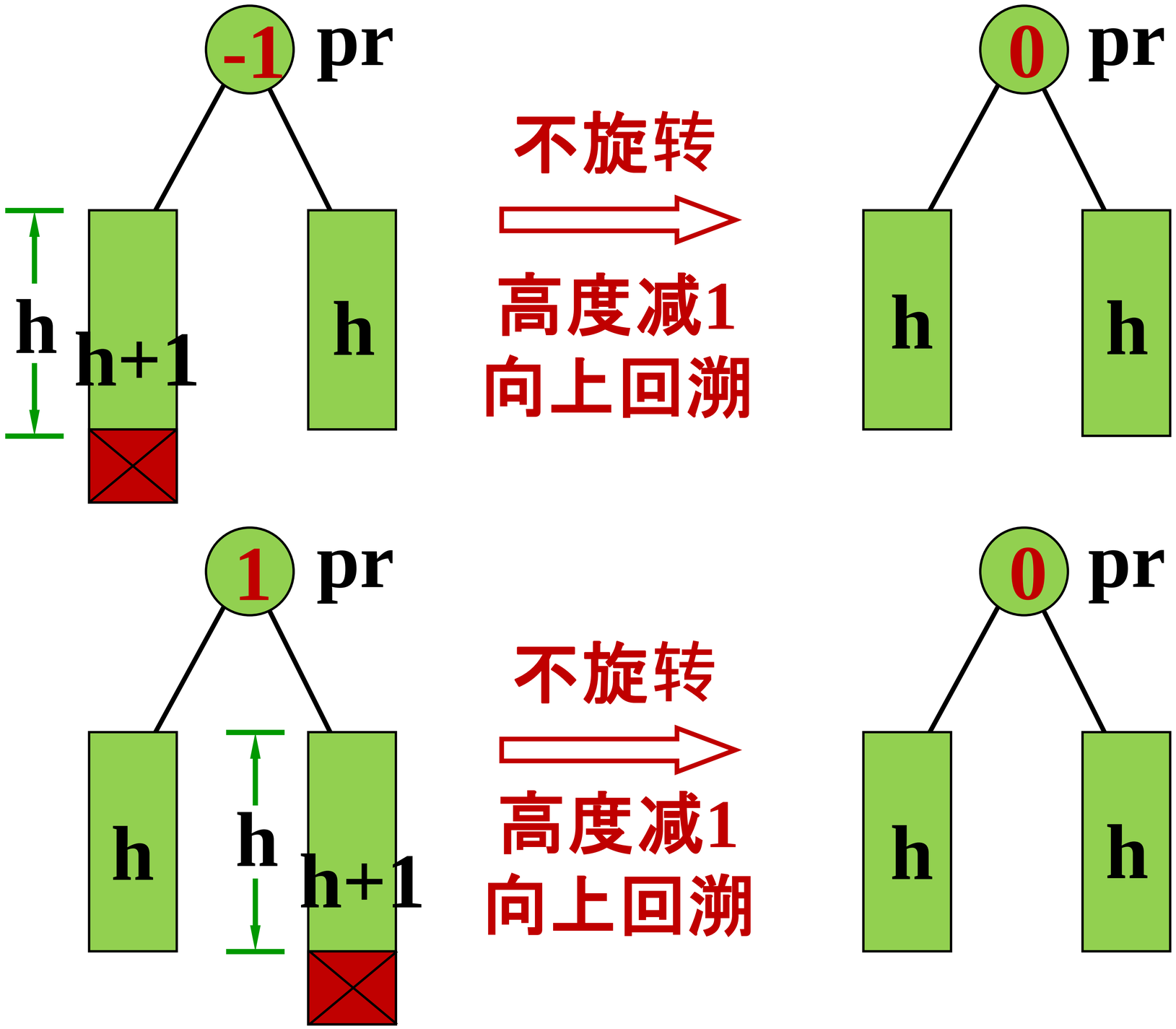
- 上述2种情况中，把最终删除的结点设为q，沿q向根的方向回溯，追踪高度变化对路径上各结点的影响
- 考察q的父结点pr，若q是其左子女，则pr的平衡因子应该加1，否则减1。修改后pr的平衡因子值有3种情况：
  - 由0改为1或者-1：删除结束(因为子树整体高度不变)
  - 由非0改为0：即较高的子树被缩短，以pr为根的子树高度减1，继续向根的方向回溯
  - 绝对值改为2：即较矮的子树又被缩短，则根据pr的平衡因子确定其较高的子树，设该子树的根为s(s一定不在q到根的路径上)
    - 根据s的平衡因子分三种情况做旋转
      - s的平衡因子为0：单旋转
      - s与pr的平衡因子同号：单旋转
      - s与pr的平衡因子异号：双旋转
    - 旋转后得到的子树的平衡因子绝对值为1，则删除结束；否则仍要向根的方向回溯



Case 1 ：pr的平衡因子由0改为1或-1

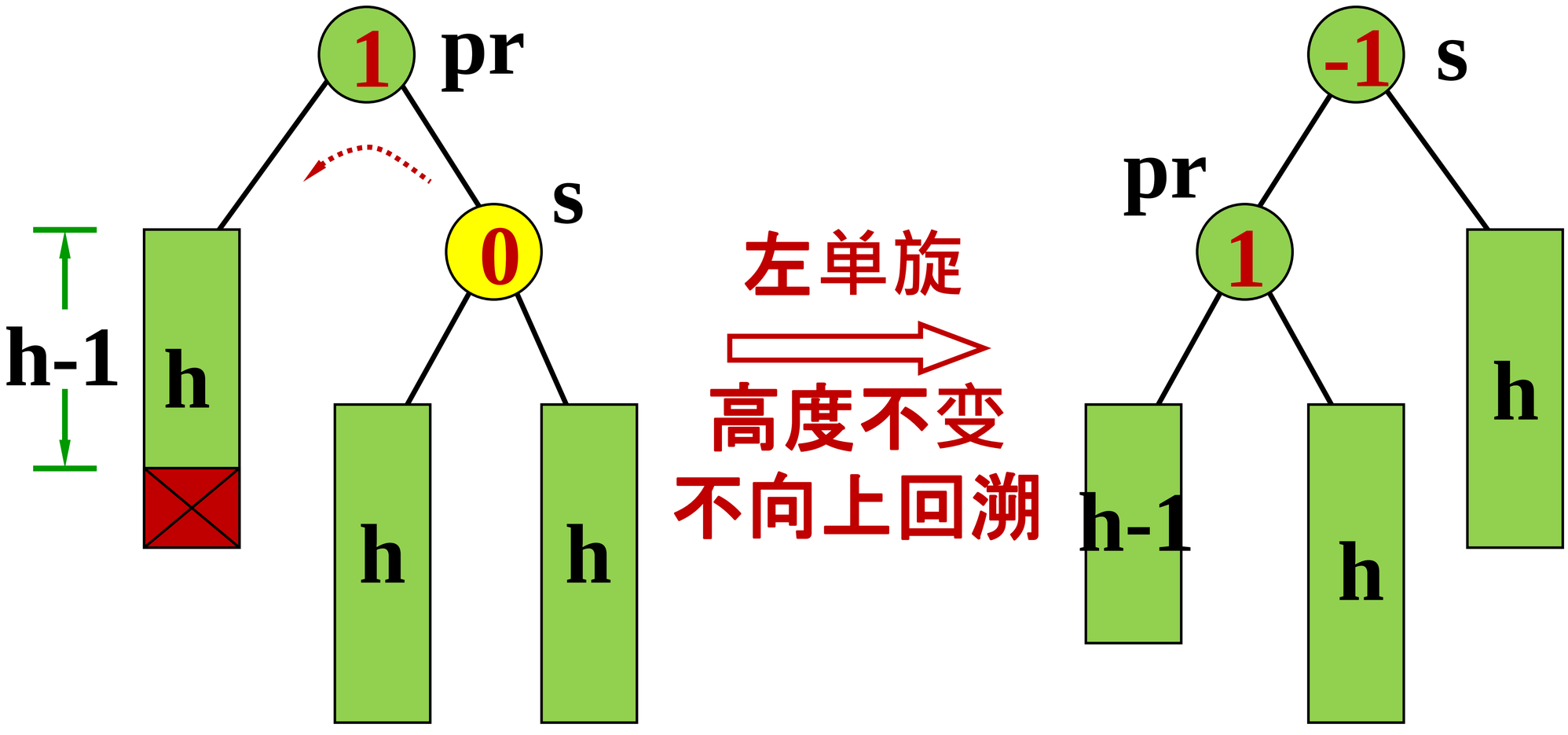


Case 2：pr的平衡因子由非0改为0



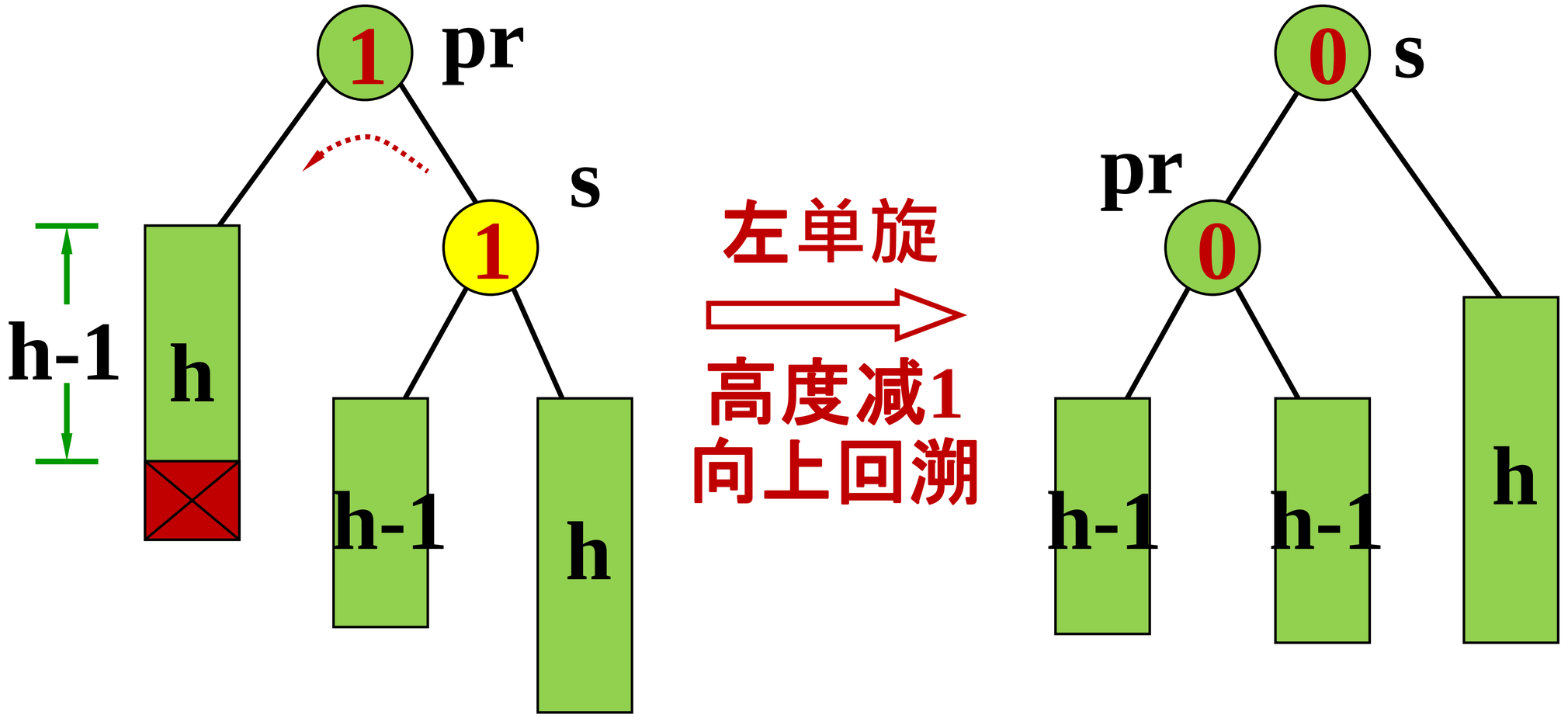
Case 3：pr的平衡因子的绝对值改为2

- Case 3a : 较高子树s的平衡因子为0



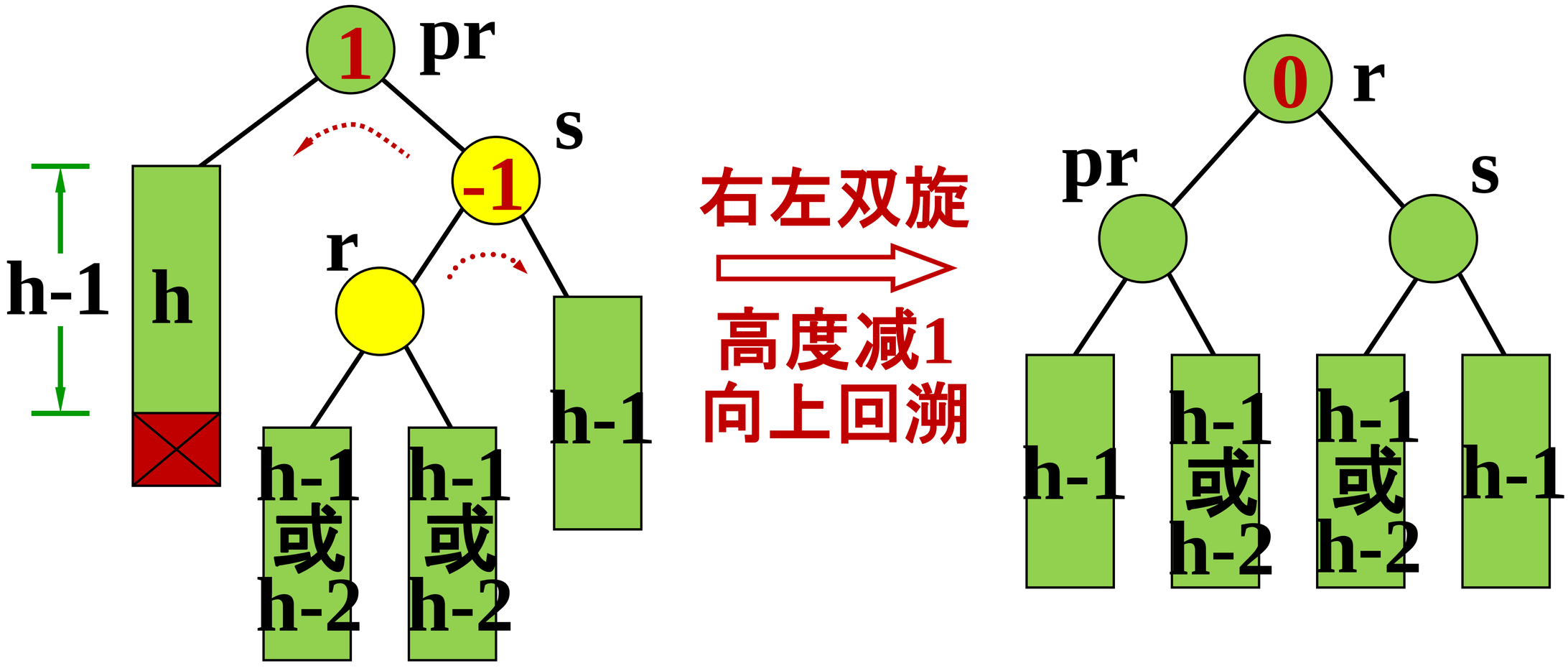
Case 3：pr的平衡因子的绝对值改为2

- Case 3b：较高子树s的平衡因子与pr的平衡因子同号



Case 3：pr的平衡因子的绝对值改为2

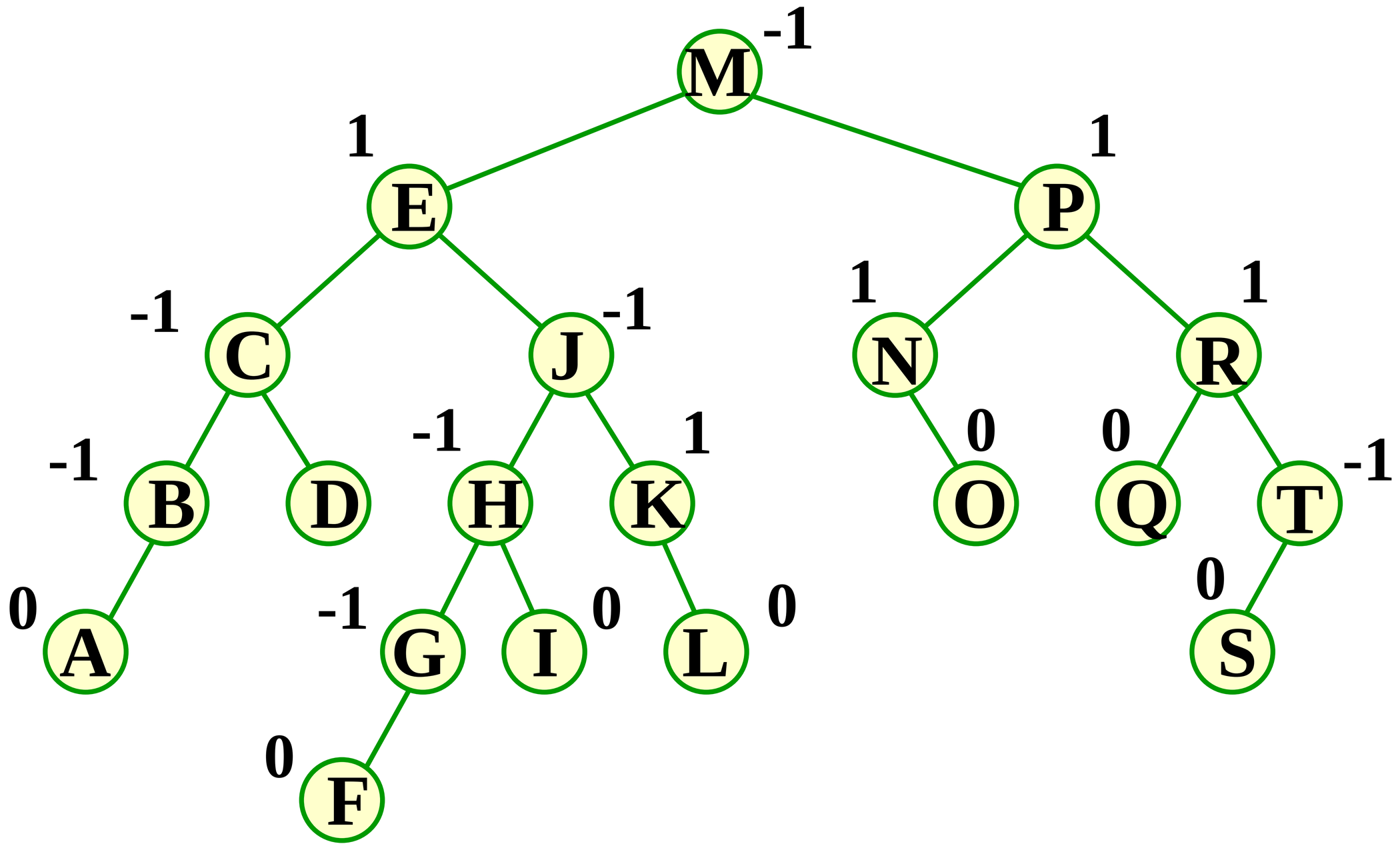
- Case 3c：较高子树s的平衡因子与pr的平衡因子异号



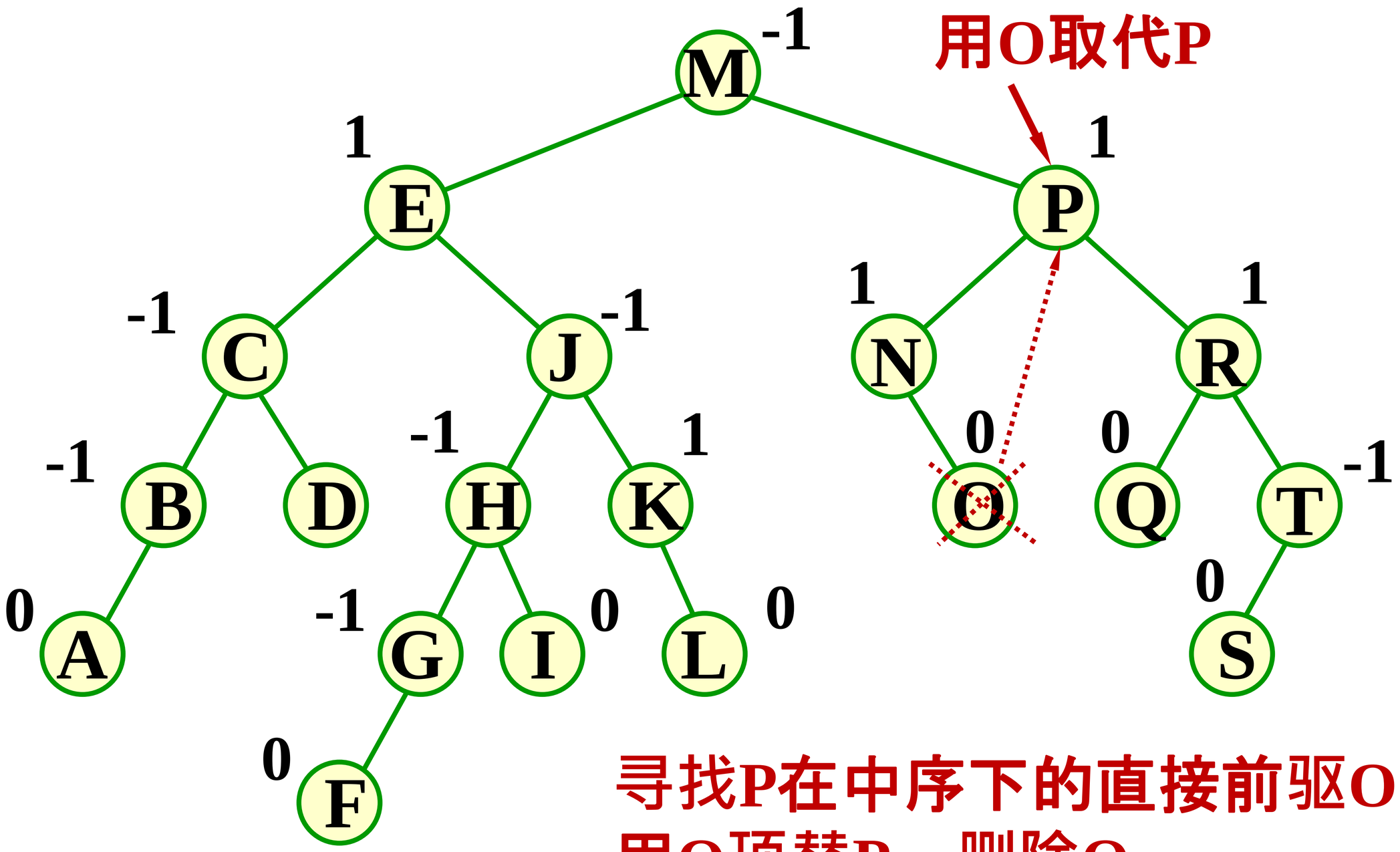
AVL树的删除算法

```
while (p != NULL)
... // 寻找删除位置，把经过的结点压栈，循环结束时，pr为被删除结点p的父结点，pr在栈顶
if (p == NULL)  return false;  // 未找到删除结点，返回
... // 处理被删除结点有两个子女的情况，转变成删除只有至多1个子女的结点的情况，最后仍然是pr为被删除结点p的父结点，pr在栈顶
if (p->left != NULL)  q = p->left; else q = p->right;
... // 让q指向p的子女结点，若被删除结点是根，则让ptr与q指向相同的结点
else
...  // 调整删除p之后的pr和q之间的连接
    while (st.IsEmpty( ) == false)
    ... // 调整pr的平衡因子，并且根据pr是其双亲ppr的左孩子还是右孩子决定dd的取值，以记住经过旋转修改的pr应该连接在ppr的左孩子还是右孩子
    if (pr->bf  == 1) || pr->bf == -1)  break;
    ... //  根据pr的平衡因子和其较高的子树的平衡因子决定旋转类型，旋转，调整平衡因子；有可能要继续向根方向回溯
        if (dd == -1)  ppr->left = pr;
        ... // 根据dd决定经过旋转修改的pr应该连接在ppr的左孩子还是右孩子
// 删除p，  结束
```

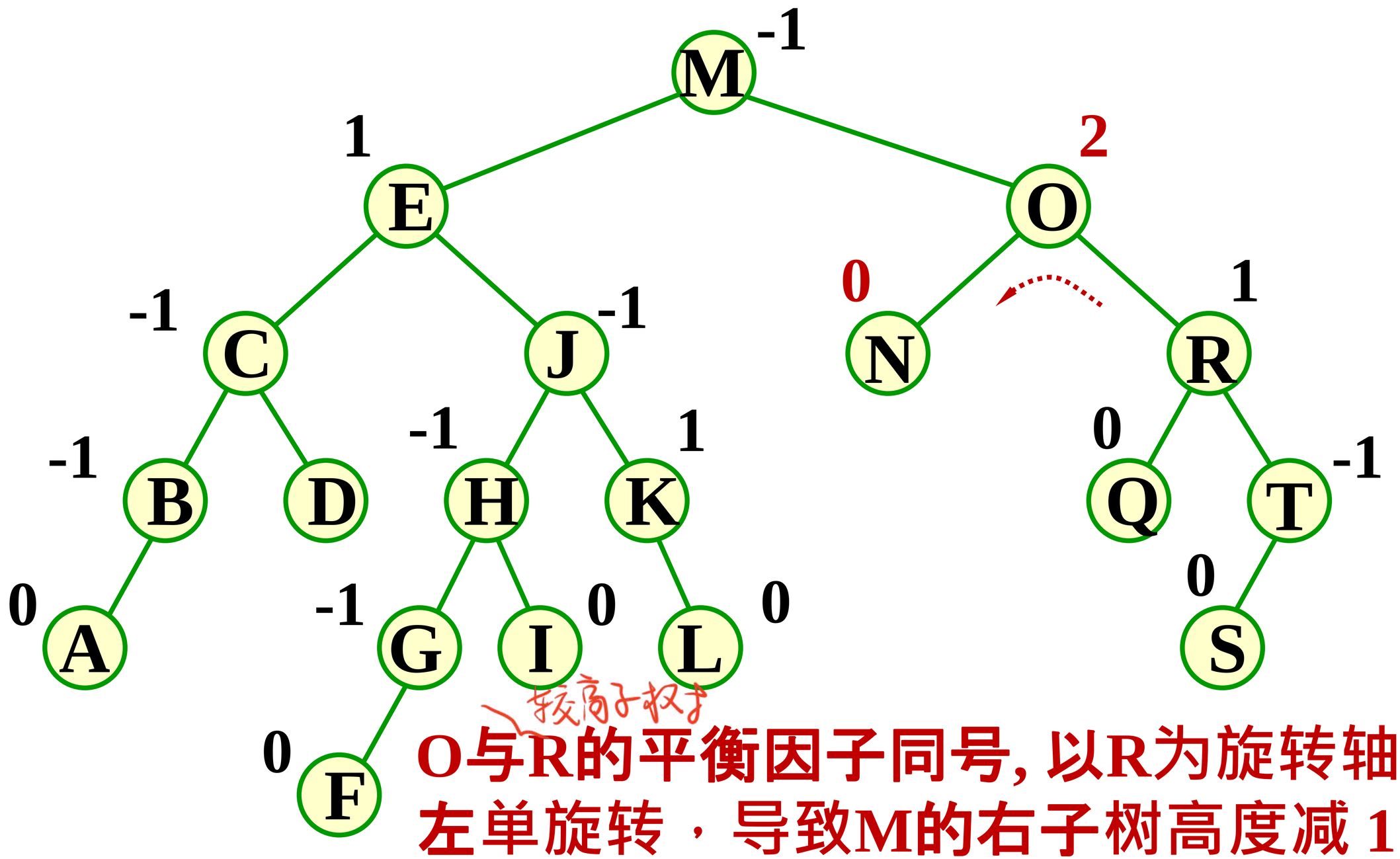
AVL树的删除示例-初始状态



AVL树的删除示例-删除结点p

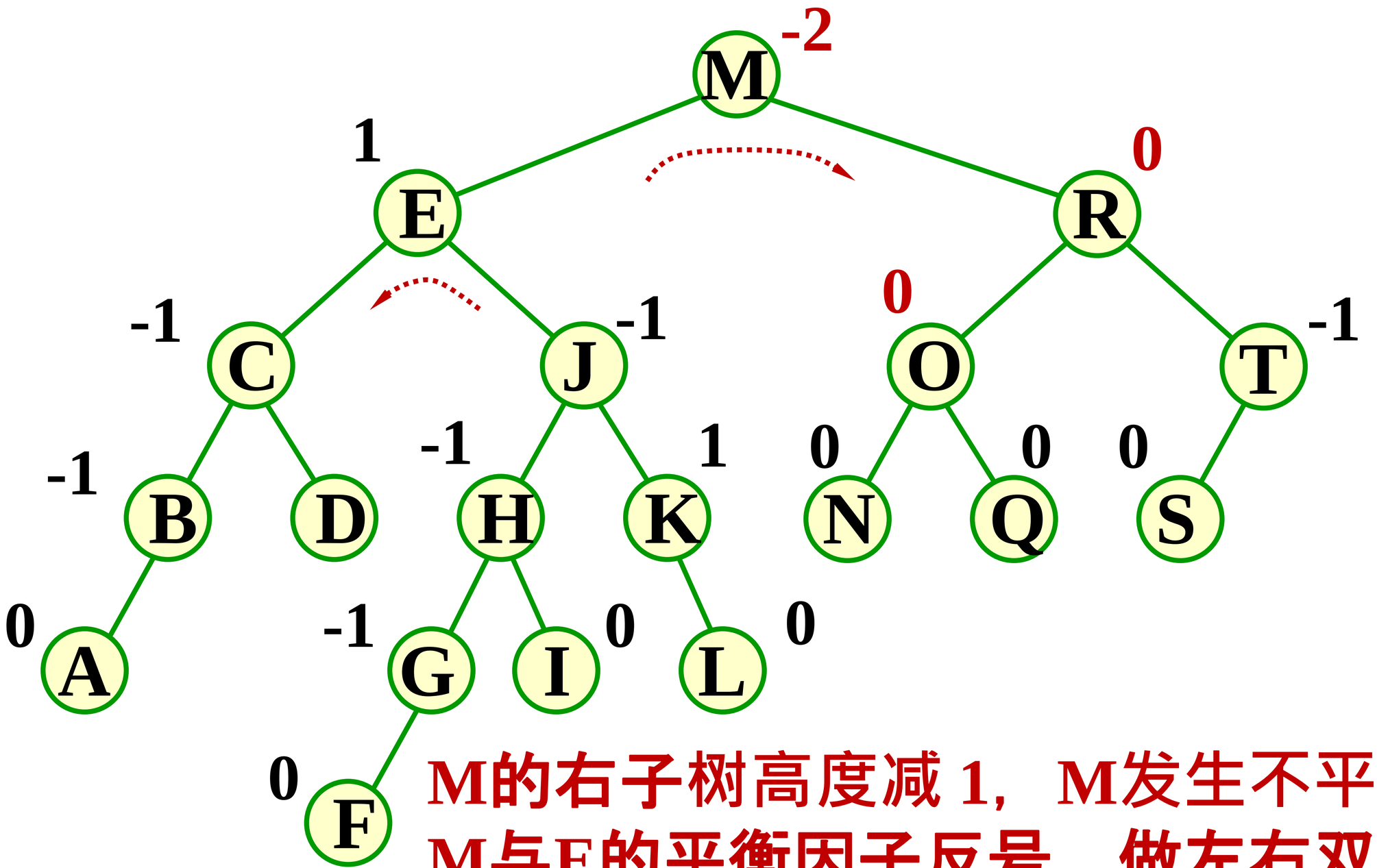


AVL树的删除示例-删除结点p



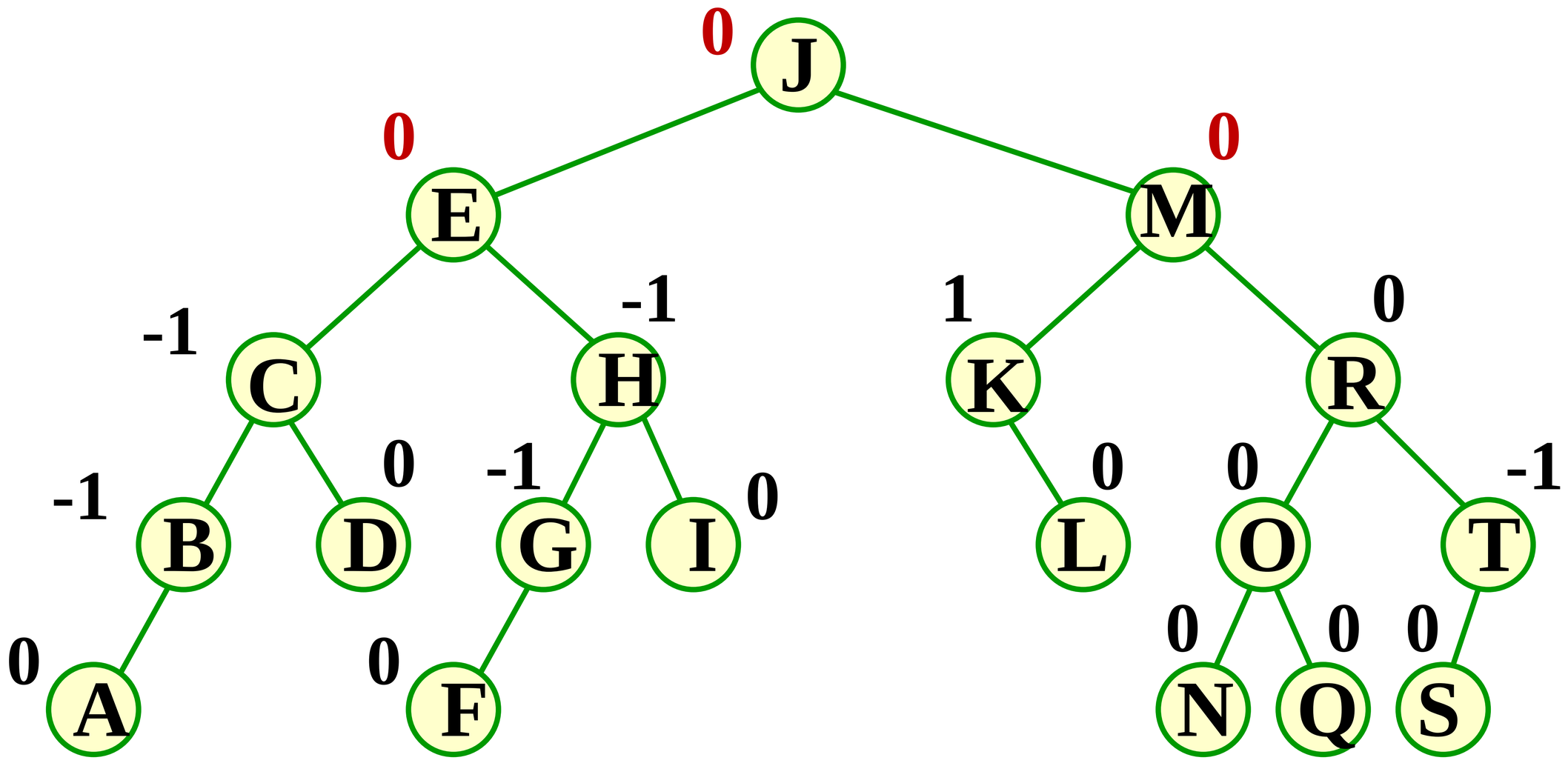
O与R的平衡因子同号, 以R为旋转轴做左单旋转，导致M的右子树高度减 1

AVL树的删除示例-删除结点p



M的右子树高度减 1，M发生不平衡，
M与E的平衡因子反号，做左右双旋转

AVL树的删除示例-删除结点p



AVL树的性能分析

- 设在新结点插入前AVL树的高度为h，结点个数为n，则插入一个新结点的时间是O(h)。对于AVL树来说，h多大？
  - 设N_h是高度为h的AVL树的最小结点数，则最差情况下根的一棵子树的高度为h-1，另一棵子树的高度为h-2，它们也是AVL树
  - 因此有$N_0=0$(空树)，$N_1=1$(仅有根结点)，$N_h=N_{h-1}+N_{h-2}+1 (h > 1)$
  - 可以证明，对于$h \geq 0$，有$N_h = F_{h+2} - 1$成立
- 有n个结点的AVL树的高度不超过$1.44*\log_2(N_h+2) - 0.328$
- 在AVL树插入和删除一个结点并做平衡化旋转所需时间复杂度为$O(\log_2 n)$

07 搜索与索引

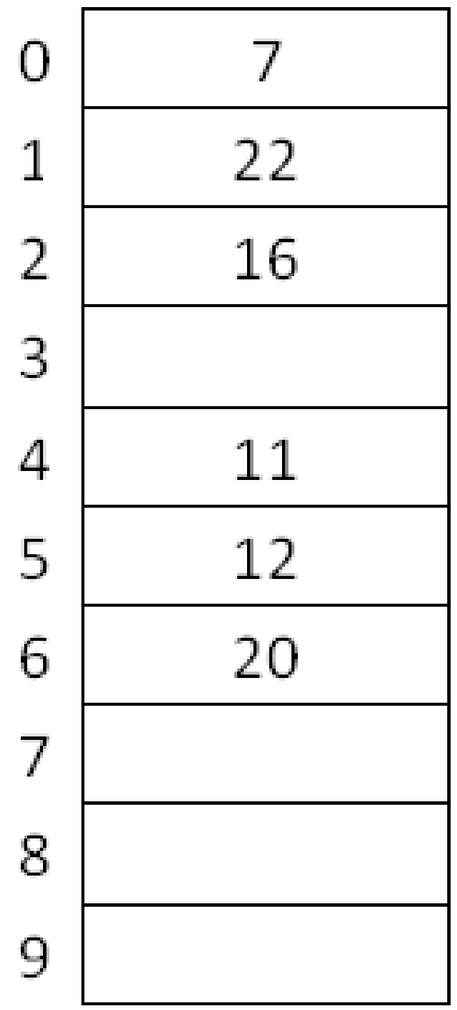
- AVL树
- **Hash搜索**

Hash搜索

- Hash，音译“哈希”或意译“散列”
- Hash搜索的前提是数据元素采用Hash表存储，即首先通过Hash函数将关键码映射到存储表中某个位置来存储数据元素，然后根据关键码用同样的方法直接访问
- Hash函数的作用是在数据元素存储地址address与其关键码key之间建立一个确定的对应关系，使每个关键码与一个唯一存储地址相对应：address＝Hash(key)
  - 在搜索时，先根据要搜索的关键码用Hash函数计算出存储地址，再按此存储地址取数据元素做比较，若关键码相等，则搜索成功
  - 在插入时，依Hash函数计算存储地址，并按此地址存放

Hash方法示例

- 含有6个结点的线性表，其中关键码分别为7, 11, 12, 16, 20, 22， 使用有10个结点的数组T[0..9]作为Hash表存储，Hash函数Hash(key)=key%7, 则Hash表的存储情况如右图所示



- 若找关键码为22的结点，则计算22%7 = 1就能在T[1]中找到

- 再插入结点8会发生什么？
  - 冲突(collision)

Hash方法的两个关键问题

- 一是选取Hash函数，要避免或尽量减少冲突
  - Hash函数是一个压缩映象函数。一般关键码集合比Hash表地址集合大得多。因此，有可能经过Hash函数的计算，把不同的关键码映射到同一个Hash地址上，这种情况称为冲突(collision)。映射到同一地址的不同关键码互为同义词(synonym)
- 二是选取解决冲突的方法

Hash函数

- 构造Hash函数时考虑的一般原则有
  - Hash函数应是简单的、能在短时间内计算出结果
  - Hash函数的定义域必须包括需要存储的全部关键码，如果Hash表允许有m个地址时，其值域必须在0到m-1之间
  - 关键码经Hash函数映射后，在地址集合中的分布概率是均等的
- Hash函数的种类很多，这里只介绍一些常见的Hash函数
  - 除留余数法
  - 直接定址法
  - 数字分析法
  - 平方取中法
  - 折叠法

除留余数法

- 取关键码被某个不大于Hash表表长m的数p除后所得余数作为Hash表存储地址
  - Hash(key) = key%p
  - 除数p一般取一个小于等于m的最大质数
- 有时候有些Hash地址是不可能用Hash函数计算出来的，只能在解决冲突时达到这些地址

除留余数法示例

- 关键码key=100，Hash表表长m=15，取质数p=13，则Hash函数为key％13，则有Hash(100)=100％13=9

- Hash表中的地址13和14不可能用Hash函数计算出

直接定址法

- 取关键码的某个线性函数值作为Hash表存储地址
  - Hash(key) = a * key + b，a和b为常数

- 这类Hash函数是一对一的映射，一般不会产生冲突。但是，它要求Hash存储地址空间的大小与关键码集合的大小相同，给应用带来限制

直接定址法示例

- 有一组关键码{942148, 941269, 940527, 941630, 941805, 941558, 942047, 940001}
- Hash函数：Hash(key) = key - 940000
- 计算出的Hash地址为
  - Hash (942148) = 2148　　Hash (941269) = 1269
  - Hash (940527) = 527　　Hash (941630) = 1630
  - Hash (941805) = 1805　　Hash (941558) = 1558
  - Hash (942047) = 2047　　Hash (940001) = 1

数字分析法

- 有n个d位数，每一位可能有r种不同的符号。这r种符号在各位上出现的频率不一定相同。根据Hash表大小，选取其中**各种符号分布均匀的若干位**作为Hash表存储地址
- **第k位符号分布均匀度**：$\lambda_k = \sum_{i=1}^{r} \left(\alpha_i^k - \frac{n}{r} \right)^2$
  - α_i^k表示第i个符号在第k位上出现的次数，n/r表示r种符号在n个数中均匀出现的期望值
  - 计算出的λ_k值越小，表明在第k位各种符号分布得越均匀
- 数字分析法仅适用于事先明确知道所有关键码每一位数值的分布情况，完全依赖于关键码集合

8个数
14个数(0~9)

$\frac{8}{10} = \frac{n}{r}$

数字分析法示例

9	4	2	1	4	8	①位，$\lambda_1 = 57.60$
9	4	1	2	6	9	②位，$\lambda_2 = 57.60$
9	4	0	5	2	7	③位，$\lambda_3 = 17.60$
9	4	1	6	3	0	④位，$\lambda_4 = 5.60$
9	4	1	8	0	5	⑤位，$\lambda_5 = 5.60$
9	4	1	5	5	8	⑥位，$\lambda_6 = 5.60$
9	4	2	0	4	7	
9	4	0	0	0	1	
①	②	③	④	⑤	⑥	

若Hash表地址为3位十进制数，应取各关键码的④⑤⑥位做为Hash表存储地址

缺点：换了一批数字之后均匀度改变

平方取中法

- 先计算构成关键码的标识符的内码的平方，然后按Hash表大小取中间若干位作为Hash表存储地址

- 设标识符可以用一个计算机字长的内码表示
  - 内码平方数的中间几位会受到标识符的所有位的影响，故对不同标识符计算的Hash表存储地址一般不会冲突

- 一般取Hash表大小为2的某次幂
  - 如8进制内码，对内码的平方数取中间的r位，则Hash表大小为8^r

平方取中法示例

标识符	8进制内码	内码的平方	Hash地址
A	01	01	001
A1	0134	20420	042
A9	0144	23420	342
B	02	04	004
DMAX	04150130	21526443617100	443
DMAX1	0415013034	526447352151420	352
AMAX	01150130	1354236617100	236
AMAX1	0115013034	345424652151420	652

折叠法

- 将关键码自左到右分成位数相等的几部分(每一部分的位数应与Hash表地址位数相同，只有最后一部分的位数可以短一些)，然后取这些部分的叠加和(舍去进位)，就可以得到具有该关键码的数据元素的Hash表存储地址
  - 移位法：把各部分的最后一位对齐相加
  - 分界法：各部分不折断，沿各部分的分界来回折叠，对齐相加

- 一般当关键码的位数很多，而且关键码每一位上数字的分布大致比较均匀时，可用该方法得到Hash表存储地址

折叠法示例

- 设给定的关键码为23938587841，若Hash表存储地址限定3位，则划分结果为每段3位，上述关键码可划分为4段：239、385、878、41

正　反　正　反

移位法		分界法	
239		239	
385		583	
878		878	
+) 41		+) 14	
1543		1714	

Hash函数的选择

- 在实际工作中应根据关键码的特点，选用适当的方法
  - 有些研究通过统计分析发现平方取中法计算的结果分布较为均匀

- 在多种Hash函数中，除留余数法不但简单，而且平均性能在很多场合优于其它类型的Hash函数
  - 是最常用的一种Hash函数

解决冲突的方法

- 冲突是不能完全避免的，两种常见的解决冲突的方法
  - 开地址法(open addressing)
  - 拉链法(chaining)

- Hash表HT有m个地址，将其设为m个桶，桶号与Hash地址一一对应，第i(0≤i<m)个桶的桶号即为第i个Hash地址
- 每个桶可存s个表项，这些表项的关键码互为同义词
  - 如果两个不同元素的关键码映射到了同一个Hash地址，就产生了冲突，它们可以放在同一个桶内的不同位置
  - 当桶内所有s个位置都放满后，再加入就会产生溢出
  - s通常取的比较小，故在桶内大多用顺序搜索，不影响性能

开地址法

- **开地址法又称闭散列法：Hash表的大小固定不变，所有的桶都直接放在Hash表数组中，每个桶只有一个表项**
- **若设Hash表中各桶的编址为0到m-1，当要加入一个元素R时，用其关键码通过Hash函数得到它的存放桶号**
- **如果在存放时发现此桶已被另一个元素占据，则发生冲突**
- **需要把R存放到Hash表中“下一个”空桶中**
- **如果Hash表未被装满，则在允许的范围内必定还有空桶**
- **可以有多种寻找“下一个”空桶的方法**
  - 线性探查法(Linear Probing)
  - 二次探查法(Quadratic Probing)
  - 双散列法

线性探查法

- 要搜索或插入一个元素时，用Hash函数计算初始桶号H_0
  - H_0 = Hash(key)
- 一旦发生冲突，在表中顺次向后寻找“下一 个”空桶H_i
  - $H_i = (H_{i-1} + 1) \% m,\ \ i = 1, 2, \ldots, m-1$
- 即用以下的线性探查序列在表中寻找“下一 个”空桶
  - $H_0 + 1, H_0 + 2, \ldots, m-1, 0, 1, 2, \ldots, H_0 - 1$
- 亦可写成如下的通项公式
  - $H_i = (H_0 + i) \% m,\ \ i = 1, 2, \ldots, m-1$

再探查 m-1次

- 当发生冲突时，探查下一个桶，循环m-1次后就会回到开始探查时的位置，说明待搜索关键码不在表内，且表已满，不能再插入新关键码

线性探查法示例

- 设一组元素的关键码为{Burke, Ekers, Broad, Blum, Attlee, Hecht, Alton, Ederly}，采用Hash函数：Hash(x) = x[0] - ‘A’(即计算关键码首字符在字母表中的位置)，设Hash表大小m为26，则线性探查法解决冲突的过程如下

Hash (Burke) = 1　Hash (Ekers) = 4　Hash (Broad) = 1　Hash (Blum) = 1

Hash (Attlee) = 0　Hash (Hecht) = 7　Hash (Alton) = 0　Hash (Ederly) = 4

0	1	2	3	4	5	6	7	8
Attlee	Burke	Broad	Blum	Ekers	Alton	Ederly	Hecht	
(1)	(1)	(2)	(3)	(1)	(6)	(3)	(1)	

平均搜索长度分析

- **用平均搜索长度衡量Hash方法的搜索性能**
  - **搜索成功的平均搜索长度ASL_{succ}**
    - 搜索到表中已有元素的平均探查次数，即找到表中各个已有元素的探查次数的平均值
  - **搜索失败的平均搜索长度ASL_{fail}**
    - 在表中搜索不到待查的元素，但找到插入位置的平均探查次数，即表中所有可能Hash到的位置上要插入新元素时为找到空桶的探查次数的平均值

包括已有元素的位置

- 对于上页的例子
  - $ASL_{succ} = 1/8 * (1 + 1 + 2 + 3 + 1 + 6 + 3 + 1) = 18/8$
  - $ASL_{fail} = 1/26 * (9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 18) = 62/26$

1+1+…+1
18个1

Hash表的类定义

```
const int DefaultSize = 100;
enum KindOfEntry { Active, Empty, Deleted };
template <class Type> class HashTable {
   public:
      HashTable( ) : TableSize(DefaultSize) { AllocateHt( ); CurrentSize = 0; }
      ~HashTable( ) { delete [ ] ht; }
      const HashTable & operator = (const HashTable &ht2);
      int Search(const Type &x);  // 搜索
      int Insert(const Type &x);  // 插入
      int Remove(const Type &x);  // 删除
      void MakeEmpty( );  // 置空
```

Hash表的类定义

```
private:
    struct HashEntry {  // 表项
        Type element;  // 表项关键码
        KindOfEntry info;  // 三种状态
        int operator == (HashEntry &);  // 判相等
        int operator != (HashEntry &);  // 判不等
        HashEntry( ) : info(Empty) { }  // 构造函数
        HashEntry(const Type& e, KindOfEntry i = Empty)
            : element(e), info(i) { }
    };
    HashEntry *ht;  // Hash表数组
    int CurrentSize, TableSize;  // 当前及最大桶数
    void AllocateHt( ) { ht = new HashEntry[TableSize]; }
    int FindPos(const Type x) const;
}
```

Hash表的解决冲突算法 - FindPos方法

```
template <class Type> int HashTable <Type> ::
FindPos (const Type x) const {
    int i =  …;  // 利用Hash函数计算初始桶号
    int j = i;
    do {
        if (ht[j].info == Empty || (ht[j].info == Active && ht[j].element == x))
            return j;  // 找到
        j = (j + 1) % TableSize;  // 循环找下一空桶
    } while (j != i);
    return j;
}
```

FindPos方法的几种返回情况

- **当FindPos方法返回某一个i，则有以下几种可能**
  - **ht[i].info == Active && ht[i].element == x**
    - 表示搜索成功，找到指定元素
  - **ht[i].info == Empty**
    - 表示未找到，但新元素可加入该位置
  - **ht[i].info != Empty && ht[i].element != x**
    - **表示Hash表已**满，不能加入新元素

Hash表的搜索算法

```
template <class Type> int HashTable <Type> ::
Search (const Type &x) {
    int i =  FindPos(x);
    if (ht[i].info != Active || ht[i].element != x)
        return -1;
    return i;
}
```

Hash表的插入算法

```
template <class Type> int HashTable<Type>::
Insert( const Type &x) {
    int i =  FindPos(x);
    if (ht[i].info != Active) {
        ht[i].element = x; ht[i].info = Active;
        CurrentSize++;
        return 1;
    }
    return 0;
}
```

Hash表的删除算法

0	1	2	3	4	5	6	7	8
Attlee	Burke	Broad	Blum	Ekers	Alton	Ederly	Hecht	

- 如果删除Broad后(即置Empty)，再搜索Blum，能否搜到？
  - 不能，会错误地判断表中没有关键码为Blum的表项

- 若想删除一个表项，只能给它做一个删除标记Deleted进行逻辑删除，不能把它真正删除
- 逻辑删除的副作用是：在执行多次删除后，表面上看起来Hash表很满，实际上有许多位置没有利用

Hash表的删除算法

```
template <class Type> int HashTable <Type> ::
Remove(const Type &x) {
    int i =  FindPos(x);
    if (ht[i].info == Active && ht[i].element == x) {
        ht[i].info = Deleted;  // 做删除标记
        CurrentSize--;
        return 1;
    }
    return 0;
}
```

线性探查法的堆积问题

- 线性探查方法容易产生**“堆积”问题，即不同探查序列的关键码占据可用的空桶，为寻找某一关键码需要经历不同的探查序列，导致搜索时间增加**

0	1	2	3	4	5	6	7	8
Attlee	Burke	Broad	Blum	Ekers	Alton	Ederly	Hecht	
(1)	(1)	(2)	(3)	(1)	(6)	(3)	(1)	

搜索Alton需要比较6次

装填因子

- Hash表的装填因子为α = n / m，n是表中已有的表项个数，m是Hash表的大小
- 可用装填因子表明Hash表的装满程度，装填因子越大，说明Hash表装得越满，发生冲突可能性越大
- 通过对线性探查法的分析，发现为搜索一个关键码所需进行的探查次数的期望值P大约是(2 - α)/(2 - 2α)，虽然平均探查次数较小，但在最坏情况下的探查次数会相当大

二次探查法

- 为改善“堆积”问题，可使用二次探查法
- 初始时，通过Hash函数得到初始桶号H_0
  - H_0 = Hash(key)
- 一旦发生冲突，寻找“下一个”空桶H_i
  - $H_i = (H_0 + i^2) \% m$, $H_i = (H_0 - i^2) \% m$, i = 1, 2, …, (m-1)/2
  - m是表的大小，它应是一个值为4k+3的质数，其中k是一个整数，如m可以是3, 7, 11, 19, 23, 31, 43, 59, 127, 251, 503, …
- 探查序列为H_0, H_0+1, H_0-1, H_0+4, H_0-4, …
- 在做$(H_0 - i^2) \% m$的运算时，当$H_0 - i^2 < 0$时，运算结果是负数，实际算式可改为$j = (H_0 - i^2) \% m$, if (j < 0) j += m

二次探查法示例

- 设一组元素的关键码为{Burke, Ekers, Broad, Blum, Attlee, Hecht, Alton, Ederly}，采用Hash函数：Hash(x) = x[0] - ‘A’(即计算关键码首字符在字母表中的位置)
- 有26个字母，所以可能的桶号是0~25，取满足4k+3的质数，表的长度m=31，则二次探查法解决冲突的过程如下

$m \geq 26$

二次探查法示例

Hash (Burke) = 1　　**Hash (Ekers) = 4**　　**Hash (Broad) = 1**　　**Hash (Blum) = 1**

Hash (Attlee) = 0　　**Hash (Hecht) = 7**　　**Hash (Alton) = 0**　　**Hash (Ederly) = 4**

+1 +1 -1 +4 -4 +1 -1

0	1	2	3	4	5
Blum	Burke	Broad		Ekers	Ederly
(3)	(1)	(2)		(1)	(2)

6	7	8	9	10	11
	Hecht				
	(1)				

25	26	27	28	29	30
		Alton			Attlee
		(5)			(3)

平均搜索长度分析

- 搜索成功的平均搜索长度
  - $ASL_{succ} = 1/8 * (3 + 1 + 2 + 1 + 2 + 1 + 5 + 3) = 18/8$
- 搜索失败的平均搜索长度
  - $ASL_{fail} = 1/26 * (6 + 5 + 2 + 3 + 2 + 2 + 20) = 40/26$
    - 这里搜索失败的平均搜索长度不是考虑总共31个表单元，而仅考虑第一次探查可能访问的26个位置
- 与之前的线性探查法相比，搜索成功的ASL相同，搜索失败的ASL则要小

算法改进

- 设第一次通过Hash函数计算出的桶号为H_0，当发生冲突时，第i-1次和第i次计算出来的“下 一个”桶号分别为

$$H_{i-1}^{(0)} = (H_0 + (i-1)^2) \% m, \qquad H_i^{(0)} = (H_0 + i^2) \% m,$$

$$H_{i-1}^{(1)} = (H_0 - (i-1)^2) \% m. \qquad H_i^{(1)} = (H_0 - i^2) \% m.$$

- 相减，可以得到

$$H_i^{(0)} - H_{i-1}^{(0)} = (2*i-1) \% m,$$

$$H_i^{(1)} - H_{i-1}^{(1)} = (-2*i+1) \% m.$$

- 从而得到

$$H_i^{(0)} = (H_{i-1}^{(0)} + 2*i-1) \% m,$$

$$H_i^{(1)} = (H_{i-1}^{(1)} - 2*i+1) \% m.$$

Hash表的解决冲突算法 - FindPos方法

```
template <class Type> int HashTable<Type> ::
FindPos ( const Type x) const {
    int i = i1 = i2 = …;  // 利用Hash函数计算初始桶号
    int k = 0, odd = 0;  // k为探查次数，odd是控制加减标志
    while ((ht[i].info == Active && ht[i].element != x) || ht[i].info == Deleted) {
        if (odd == 0) {  // odd == 0，  加i²
            k++;
            i = i1 = (i1 + 2 * k - 1) % TableSize; odd = 1;
        } else {  // odd == 1，  减i²
            i2 = (i2 - 2 * k + 1) % TableSize; odd = 0;
            if (i2 < 0) i2 = i2 + TableSize;
            i = i2;
        }
    }
    return i;
}
```

算法特点

- 根据相关证明，当表的长度TableSize为质数且表的装填因子α不超过0.5时，新的表项一定能够插入，且任何一个位置不会被探查两次。只要表中至少有一半空，就不会有表满问题
- 在插入一个表项时，必须确保表的装填因子α不超过0.5，如果超出，必须将表长度扩充一倍
  - 然后把原来表中的元素按新的Hash函数逐一计算重新插入新表中
  - 这样，在搜索时就可以不考虑表满的情况
- 在删除一个表项时，为确保搜索链不致中断，也只能做表项的逻辑删除，即将被删元素的info改为Deleted

双散列法

- 双散列法需要两个Hash函数
  - 第一个Hash函数按表项的关键码计算得到桶号H_0 = Hash(key)
  - 一旦发生冲突，利用第二个Hash函数计算到达“下一个”桶的移位量p = ReHash(key)，即下一个桶号为(H_0 + p) % m，其中m是Hash表的大小，p是小于m且与m互质的整数 i*p
  - 探测序列为H_i = (H_0 + i * ReHash(key)) % m，i = 1, 2, …, m-1
  - 按一定的距离跳跃式地寻找“下一个”桶，减少了堆积的机会
  - 最多经过m-1次探查，它会遍历表中所有位置，回到H_0位置

双散列法示例

- 给出一组关键码{22, 41, 53, 46, 30, 13, 01, 67}
- 第一个Hash函数为：Hash(x)＝(3x) % 11
- Hash表为HT[0..10]，　m = 11
  - 第二个Hash函数为：ReHash(x) = (7x) % 10 +1
    - 故有$H_i = (H_{i-1} + (7x) \% 10 + 1) \% 11, i = 1, 2, \ldots$

$H_0(22) = 0$　$H_0(41) = 2$　$H_0(53) = 5$　$H_0(46) = 6$　$H_0(30) = 2 \rightarrow H_1 = (2+1) = 3$

$H_0(13) = 6 \rightarrow H_1 = (6+2) = 8$　$H_0(01) = 3 \rightarrow H_1 = (3+8) = 0 \rightarrow H_2 = (0+8) = 8$

$\rightarrow H_3 = (8+8) = 5 \rightarrow H_4 = (5+8) = 2 \rightarrow H_5 = (2+8) = 10$

$H_0(67) = 3 \rightarrow H_1 = (3+10) = 2 \rightarrow H_2 = (2+10) = 1$

0	1	2	3	4	5	6	7	8	9	10
22	67	41	30		53	46		13		01
(1)	(3)	(1)	(2)		(1)	(1)		(2)		(6)

平均搜索长度分析

- **搜索成功的平均搜索长度**
  - **ASL_{succ} = 1/8 * (1 + 3 + 1 + 2 + 1 + 1 + 2 + 6) = 17/8**
- **搜索失败的平均搜索长度**
  - **根据key的不同，每一Hash地址的移位量(即p = ReHash(key))有10种：1, 2, …, 10。先计算每一Hash地址各种移位量情形下找到下一个空位的比较次数，并求出平均值** 因为key未知
  - **再计算各个Hash地址的平均比较次数的总平均值**

ReHash()的取法

- 有很多做法，如
  - 当m是质数时，可定义
    - ReHash(key) = key % (m-1) + 1
    - ReHash(key) = ⌊key / m⌋ % (m-2) + 1
  - 当m是2的方幂时，ReHash(key)可取从0到m-1中的任意一个奇数

开地址法的缺点

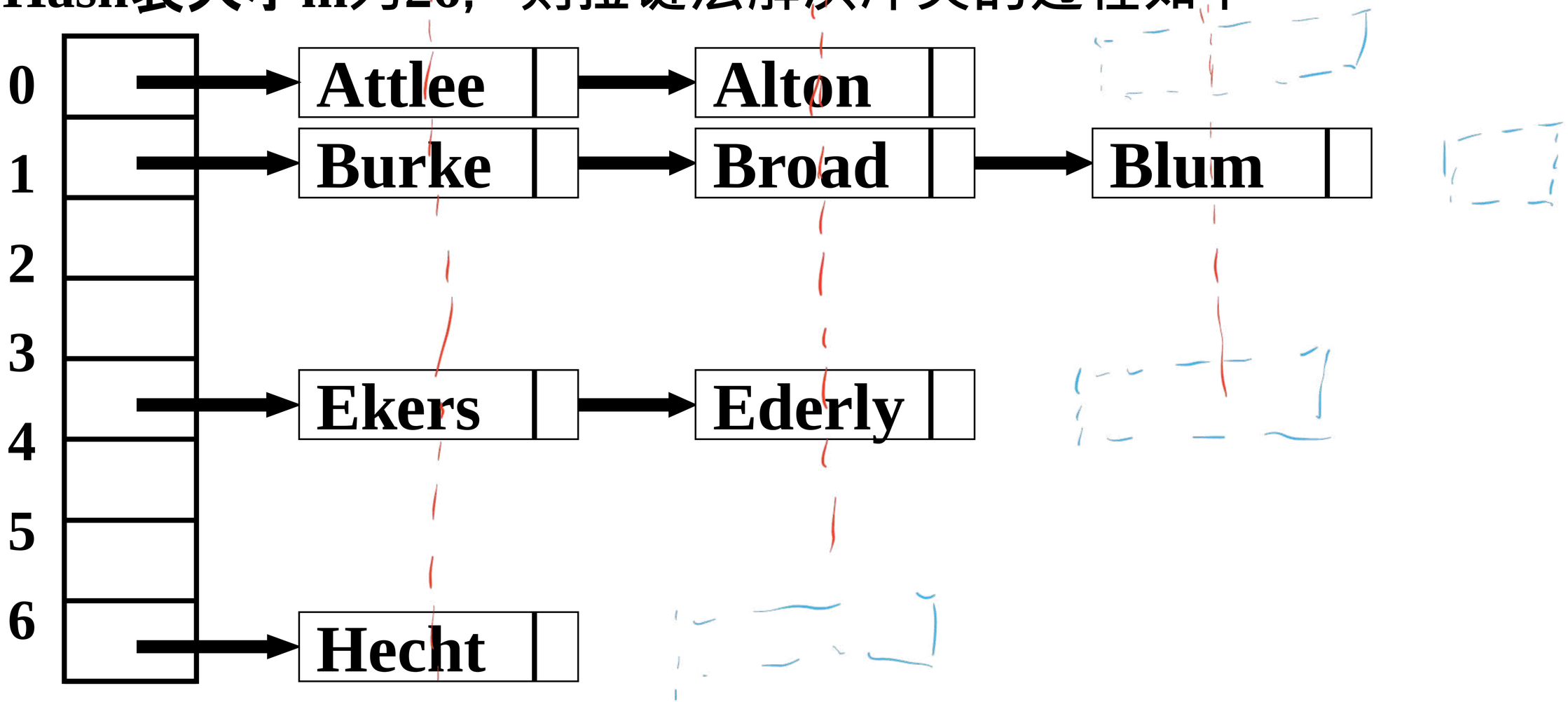
- 由于在闭散列情形下若想删除一个表项，只能给它做一个删除标记Deleted进行逻辑删除，不能把它真正删除
- 逻辑删除的副作用是：在执行多次删除后，表面上看起来Hash表很满，实际上有许多位置没有利用
- 因此当Hash表经常变动时，最好不用闭散列方法解决冲突，可改用拉链法(即开散列法)来解决冲突

拉链法

- 拉链法(又称为开散列法、链地址法)
  - 对关键码集合用某一个Hash函数计算它们的存放位置
  - 设Hash表地址空间的位置是从0到m-1，则关键码被划分为m个子集，具有相同地址的关键码归于同一子集，每个子集称为一个桶
  - 每个桶中的元素用一个单链表链接起来，称之为同义词子表，所有桶号相同的元素都链接在同一个同义词子表中
  - 各链表的表头结点组成一个数组，数组的元素个数与桶数一致，桶号为i的同义词子表的表头结点是数组中的第i个元素

拉链法示例

- 设一组元素的关键码为{Burke, Ekers, Broad, Blum, Attlee, Alton, Hecht, Ederly}，采用Hash函数：Hash(x) = x[0] - ‘A’(即计算关键码首字符在字母表中的位置)，设Hash表大小m为26，则拉链法解决冲突的过程如下



平均搜索长度分析

- **搜索成功的平均搜索长度**
  - **ASL_{succ} = 1/8 * (1 * 4 + 2 * 3 + 3 * 1) = 13/8**

- **搜索失败的平均搜索长度**
  - **ASL_{fail} = 1/26 * (3 + 4 + 3 + 2 + 22) = 34/26**



- **通常，每个桶中的同义词子表都很短，设有n个关键码通过某一个Hash函数，存放到Hash表中的m个桶中，那么每一个桶中的同义词子表的平均长度为n/m，以搜索平均长度为n/m的同义词子表代替了搜索长度为n的顺序表，搜索速度快得多**

Hash表的解决冲突算法 - FindPos方法

```
template<class Type> ChainNode<Type> * HashTable<Type> ::
FindPos (const Type x) {
    int j = …; // 利用Hash函数计算初始桶号
    ChainNode<Type> * p = ht[j];
    while(p != NULL && p->data != x) p = p->link;
    return p;
}
```

Hash搜索的讨论

- 冲突很难避免，增加搜索时间，这与Hash函数的选取(地址分布是否均匀)、解决冲突的方法(是否产生堆积)有关
- 要考虑到可能增加冲突的因素，如关键码是否具有相同的前缀或后缀、是否有相同字符的不同排列的情况、...
- 当装填因子α较高时，选择的Hash函数不同，Hash表的搜索性能差别很大
- 对Hash表技术进行的实验评估表明，它具有很好的平均搜索性能，优于一些传统的技术(如AVL树)
- 但在最坏情况下性能不好，对一个有n个关键码的Hash表执行搜索或插入操作，最坏情况下需要 O(n)的时间

一些实验结果

$\alpha = n / m$	0.50		0.75		0.90		0.95	
散列函数种类	开散列法	闭散列法	开散列法	闭散列法	开散列法	闭散列法	开散列法	闭散列法
平方取中	1.26	1.73	1.40	9.75	1.45	310.14	1.47	310.53
除留余数	1.19	4.52	1.31	10.20	1.38	22.42	1.41	25.79
移位折叠	1.33	21.75	1.48	65.10	1.40	710.01	1.51	118.57
分界折叠	1.39	22.97	1.57	48.70	1.55	69.63	1.51	910.56
数字分析	1.35	4.55	1.49	30.62	1.52	89.20	1.52	125.59
理论值	1.25	1.50	1.37	2.50	1.45	5.50	1.48	10.50

搜索关键码时所需对桶的平均访问次数

开地址法和拉链法比较

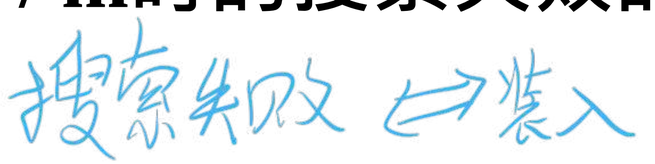
- 研究表明，拉链法的ASL一般小于开地址法，故得到更广泛的应用，与开地址法相比，拉链法有如下优点
  - 拉链法解决冲突时，非同义词决不会发生冲突，ASL较短
  - 拉链法中链表结点空间是动态申请的，适合无法确定表长的情况
  - 开地址法为减少冲突，要求装填因子α较小，故当结点规模较大时会浪费很多空间。而拉链法中可取α≥1，且结点规规模较大时，拉链法中增加的指针域可忽略不计，因此节省空间
  - 在用拉链法构造的Hash表中，删除结点的操作易于实现，只要简单地删去链表上相应的结点即可。而对开地址法构造的Hash表，删除结点时只能在被删结点上做删除标记，不能真正删除结点

平均搜索长度

- 设α是Hash表的装填因子

$$\alpha = \frac{\text{表中已装有记录数}}{\text{表中预设的最大记录数}} = \frac{n}{m}$$

- 在长度为m的Hash表中已装入n个元素的情况下
  - S_n是搜索一个随机选择的关键码$x_i(1 \le i \le n)$所需执行的关键码比较次数的期望值
  - U_n是装入第n+1个元素所需执行的关键码比较次数的期望值
- 前者称为在α = n / m时的搜索成功的平均搜索长度，后者称为在α = n / m时的搜索失败的平均搜索长度



用不同的方法解决冲突时Hash表的平均搜索长度

处理冲突的方法		平均搜索长度 *ASL*	
		搜索成功 *Sn*	搜索不成功 *Un* (登入新记录)
闭散列法	线性探查法	$\frac{1}{2}\left(1+\frac{1}{1-\alpha}\right)$	$\frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right)$
	伪随机探查法 二次探查法 双散列法	$-\left(\frac{1}{\alpha}\right)\log_e(1-\alpha)$	$\frac{1}{1-\alpha}$
开散列法 (同义词子表法)		$1+\frac{\alpha}{2}$	$\alpha+e^{-\alpha}\approx\alpha$

关于装填因子的进一步说明

- **Hash表的装填因子α表明了表中的装满程度**
  - **越大，** 说明表越满，再插入新元素时发生冲突的可能性就越大
- **Hash表的搜索性能，即平均搜索长度依赖于Hash表的装填因子，不直接依赖于n或m**
  - **不论表的长度有多大，总能选择一个合适的装填因子，以把平均搜索长度限制在一定范围内**

练习

- 设有一个含200个表项的Hash表，用二次探查法解决冲突，按关键码搜索时找到一个新表项插入位置的平均探查次数不超过1.5，则Hash表的大小应能够至少容纳多少个表项？设搜索失败的平均搜索长度为$U_n = 1/(1-\alpha)$
- 由于$U_n = 1/(1-\alpha) \leq 1.5$，故$\alpha \leq 1/3$
- 由于$\alpha = n / m = 200 / m \leq 1/3$，故$m \geq 600$
- 由于采用二次探查法，m应该是一个4k+3的质数，故可以取604+3 = 607

本章小结

- **AVL树**
- **Hash搜索**

作业

- 有一个关键码的输入序列{55, 31, 11, 37, 46, 73, 63, 02, 07}
  - 从空树开始构造AVL树，画出每插入一个结点时的二叉树，若发生不平衡，指明需要的平衡旋转类型以及平衡化后的AVL树
  - 计算最终的AVL树在等概率情况下搜索成功/失败的平均搜索长度
- 设Hash表为HT[13]，Hash函数为Hash(key) = key%13，用闭散法解决冲突，对关键码12, 23, 45, 57, 20, 03, 78, 31, 15, 36依次插入Hash表
  - 计算采用线性探查法时的Hash表，计算等概率下搜索成功/失败的平均搜索长度
  - 计算采用双散列法时的Hash表，ReHash(key) = (7*key)%10+1，计算等概率下搜索成功的平均搜索长度
- 有150个记录要存储到Hash表中，并用线性探查法解决冲突，要求找到所需记录的平均比较次数不超过2次，那Hash表需要多大？设a是Hash表的装填因子，$ASL_{succ} = (1+1/(1-a))/2$